

Programmation Orientée Objet

avec: C++ et JAVA

SMI / S5

Année universitaire: 2021/2022

Pr. Jaouad BOUMHIDI
Département d'Informatique

Les concepts clés de la POO

- Classe et Objets
- L'encapsulation
- L'héritage
- Le polymorphisme

Intérêt de la POO

- **Les programmes sont plus clairs**
- **Sécurité des données**
- **La maintenance des applications est facilitée**
- **Il est facile de faire évoluer des programmes**
- **Le code est facilement réutilisable**

Partie1

Programmation Orientée Objet avec: C++

- **SPECIFICITES DU C++**
- **Notions de base de Programmation Orientée Objet :
Classes, Constructeurs et destructeurs d'objets,
Propriétés des méthodes**
- **Fonctions Amies et Surdéfinition des Opérateurs**
- **Héritage simple**
- **Fonctions virtuelles et Polymorphisme**
- **Héritage multiple**

SPECIFICITES DU C++

C++ = C + Orientée Objet

Cependant:

Il existe des incompatibilités avec le langage C

Déclaration de Variables:

En C++, il n'est plus nécessaire de regrouper les déclarations en début de fonction ou en début de bloc.

Il devient aussi possible d'employer des expressions dans des initialisations

Exemples:

```
int n ;
```

```
....
```

```
n=... ;
```

```
....
```

```
int q=2*n-1 ;
```

```
....
```

```
for (int i=0 ;i<n ;i++)      //      en C: int i; for (i=0 ;i<n ;i++)...
```

Initialisation de variable lors de sa déclaration

Deux notations sont possibles :

int p=34 ; équivalent à int p (34) ;

Entrées / Sorties standard

Les fonctions du C, définies dans `<stdio.h>` *restent tjrs valables*.
C++ offre des alternatives plus conviviales, selon `<iostream.h>`

cin : flot lié à l'entrée standard, par défaut le clavier

cout : flot lié à la sortie standard, par défaut l'écran

Deux opérateurs sont définis :

>> Cet opérateur extrait une variable ou une expression du flot d'entrée

<< Cet opérateur envoie une variable ou une expression sur le flot de sortie

*Ces opérateurs agissent sur tous les types prédéfinis. Il n'est pas nécessaire, comme c'était le cas avec les fonctions du C, de préciser les types des variables (**les codes formats**).*

Syntaxe:

cout<<expression1<<expression2<<.....<<expressionN;

affiche sur l'écran les valeurs des différentes expressions.

cin>>lvalue1>>lvalue2>>.....>>lvalueN

lit sur le flot cin les informations de l'un des types prédéfinis: **char, short, int, long, float, double , ou char***

Le caractère fin de ligne peut être inséré par le manipulateur **endl**, prédéfini dans le fichier <iostream.h>.

Exemple: **cout<<" Bonjour" <<endl;**

Remarque :

La norme utilise aussi le nom `iostream`, sans extension. Cependant, il faut faire appel à la notion d'espace de nom selon la syntaxe suivante :

```
#include<iostream>  
using namespace std ;
```

Exercice

Ecrire le programme suivant (correcte en C et en C++) en utilisant les nouvelles possibilités d'entrées/sorties de C++ (en évitant l'utilisation des fonctions printf, scanf) :

```
#include<stdio.h>  
main(){  
int n ; float x ;  
printf("Entrer un entier et un float\n ");  
scanf("%d %f ",&n,&x) ;  
printf("le produit de %d par %f est : %f ",n,x,n*x) ;  
}
```

Types

- **Type booléen:**

Le type est *bool* et les valeurs sont *true* et *false*

- **Types dérivés**

En C++, on définit comme en C des structures. Mais, dans la déclaration d'objets de ces types dérivés, les mots clés **struct**, ne sont plus nécessaires ;

- **Exemple :**

```
struct assure{...} ;
```

```
main() {  
  const int n=50 ;  
  assure a[n] ; //en C struct assure a[n] ;  
}
```

Fonctions: Arguments par défaut:

Possible de prévoir dans le prototype, pour un ou plusieurs arguments des valeurs par défaut

Exemple :

```
float fct( char,int=10,float=0.0) ;
```

Au niveau de l'appel :

fct('a') sera équivalent à fct('a',10, 0.0) ;

fct('x',12) sera équivalent à fct('x', 12, 0.0).

Passage d'arguments par référence :

Solution hybride entre le passage par valeur et par pointeur.

Exemple: Permutation de deux valeurs :

```
void échange(int& a, int& b){  
    int t=a ;  
    a=b ;  
    b=t ;}
```

```
main() {  
    int x=12, y=5;  
    échange(x,y);  
    cout<< "x= "<<x<< " "<< "y= "<<y<<endl ;  
}
```

Surcharge de fonctions :

En C++, on peut définir des fonctions à nom générique, c'est-à-dire que, à un nom de fonction sont associées plusieurs définitions qui diffèrent par les types et le nombre de leurs arguments. A l'appel, le compilateur se base sur la liste des arguments effectifs pour choisir la définition appropriée.

Exemple :

```
float test(int i , int j)  
{return float(i+j);}
```

```
float test(float i , float j)  
{return i*j;}
```

```
int main(){  
float x=2.5,y=3.5;  
cout<<test(x,y);}
```

Gestion dynamique de la mémoire

- En plus des fonctions malloc et free du C, le C++ fournit d'autres moyens pour allouer et libérer la mémoire :

new, delete, new[] et delete[].

Les deux opérateurs new et new[] permettent d'allouer de la mémoire et les deux opérateurs delete et delete[] de la libérer :

- **Exemple :**

```
int *a ;
```

```
a=new int ;
```

```
delete a;
```

- ***new type[n]*** : alloue la place pour n éléments du type spécifié et renvoie l'adresse du premier octet du tableau alloué
- ***delete [] pointeur*** : permet la libération d'un tableau d'objets

- **Exemple :**

```
int *a =new int[20] ;
```

```
delete [ ] a;
```

Exercice

Ecrire le programme suivant (correcte en C et en C++) en utilisant les nouvelles possibilités d'entrées/sorties de C++ (en évitant l'utilisation des fonctions printf, scanf et malloc) :

```
#include<stdio.h>  
main(){  
int n,*a ; float x,*b ;  
printf("Entrer un entier et un float\n ");  
scanf("%d %f ",&n,&x) ;  
printf("le produit de %d par %f est : %f ",n,x,n*x) ;  
a=malloc(sizeof(int)) ;  
b=malloc(sizeof(float)*50) ;}
```

Cas d'échec d'allocation

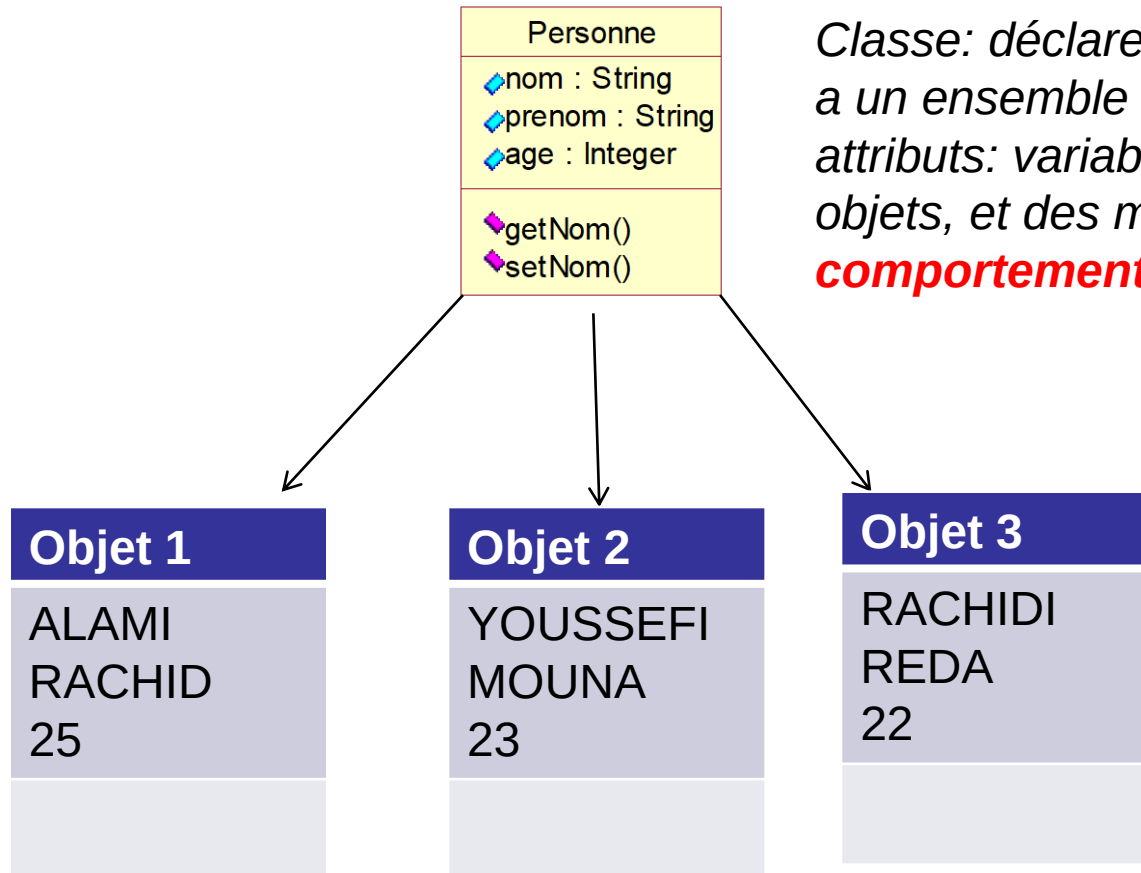
- choisir entre le renvoi d'un pointeur de valeur nulle

Ou:

- L'interruption de l'exécution du programme et le déclenchement d'une exception *bad_alloc*. (comportement du gestionnaire installé par défaut).

Notions de base de Programmation Orientée Objet :
Classes, Constructeurs et destructeurs d'objets,
Propriétés des méthodes

Concept de classe et Objet



*Classe: déclare des propriétés communes a un ensemble d'objets. Elle définit des attributs: variables représentant **l'état** des objets, et des méthodes définissant leurs **comportements**.*

*Objet=Instance de classe
Dont les attributs ont une valeur*

Le concept de classe :

Déclaration d'une classe:

Une classe est déclarée en dehors de toute fonction.
Le plus souvent elle constitue un fichier séparé.

```
class nom-classe {  
    ... déclaration des membres données  
    ... déclaration des membres fonctions (prototypes)  
};
```

Chaque déclaration peut être précédée d'un statut :

- ***private*** mot clé pour rendre la donnée ou la fonction inaccessible en dehors de la classe
- ***public*** mot clé pour la rendre accessible

Exemple :

On considère la classe *point* permettant de manipuler un point dans le plan et qui comporte deux membres données réels privés nommés *abs* et *ord* et trois fonctions membres publiques *initialise*, *deplace* et *affiche* :

//Déclaration de la classe point

```
class point {  
    // déclaration des membres privés  
    private :  
    float abs ;  
    float ord ;  
    //déclaration des membres publiques  
    public :  
    void initialise(float, float) ;  
    void affiche() ;  
} ;
```

Définition d'une classe

consiste à fournir les définitions des fonctions membre qui peuvent être définies à l'intérieur de la classe, Comme elles peuvent aussi être définies en dehors de la classe :

Définition de la fonction initialise à l'intérieure de la classe *point* :

```
class point{  
private :  
float abs ;  
float ord ;  
public :  
void initialise(float x, float y){abs=x ; ord=y ;}  
void affiche() {cout<<abs<<endl ; cout<<ord<<endl;}  
};
```

Définition de la fonction initialise en dehors de la classe :

```
class point{  
private :  
float abs ;  
float ord ;  
public :  
void initialise(float, float);  
void affiche() ;  
};
```

Syntaxe :

```
Type-retour nom-class :: nom-fonction(argument) {  
    . . . instructions  
}
```

```
void point :: initialise(float x, float y){abs =x; ord=y ;}
```

Utilisation d'une classe et instantiation d'un objet:

Nous allons maintenant créer « **l'objet** » d'un type class

Nom_classe a ;

Exemple :

point a,b ;

On peut accéder à n'importe quel membre public (donnée ou fonction) d'une classe en utilisant l'opérateur .

syntaxe: **a.nom-fonction (arguments);**

Exemple :

a.initialise(4, 1) ;

Constructeurs et destructeurs d'objets

- Quand un objet est instancié à partir d'une classe, une fonction appelée constructeur associé à cette classe est automatiquement exécutée.

Si le programmeur n'a pas ajouté constructeur explicitement, le constructeur par défaut est implicitement et automatiquement appelé. (permet la création et l'initialisation de l'objet par: Zéro pour les float, double et int . Par le caractère nulle pour les caractères et par false pour les boolean)

Le constructeur ajoutée explicitement a pour nom le nom de la classe. Elle ne comporte pas de type retour (ce n'est pas une fonction qui renvoie un argument de type void). Elle peut recevoir zéro, un ou plusieurs arguments

Déclaration et définition d'un constructeur

Un constructeur de la classe point pourrait se déclarer explicitement par :

public:
point(float,float) ;

Et être défini en dehors de la classe par :

point :: point(float x, float y){
abs=x; ord=y; }

- ***L'instanciation prend la forme :***
point a(x,y) ;

Constructeur avec arguments par défaut

Point (float x=0. , float y=0.) {abs=x ; ord=y ; }

Dans ce cas on peut déclarer dans la fonction utilisatrice :

point a(5.5 , -2.) ; un point de coordonnées 5.5 et -2. est créé

point b; un point de coordonnées 0. et 0. est créé

point c(3.) ; un point de coordonnées 3. et 0. est créé

Destructeurs

Un destructeur de la classe point pourrait se déclarer par :

public:

~point () ;

Et se défini en dehors de la classe par :

point : : ~point() { ... }

Exercice 1 :

Créer une classe vect3d pour représenter un vecteur dans l'espace caractérisés par ses coordonnées x, y et z :

Et comporte un constructeur et une méthode d'affichage

Écrire un programme principal de test qui crée un vecteur et l'affiche

Propriétés des méthodes

- **Surcharge des méthodes**

Une méthode peut être surchargée comme toute fonction ordinaire en C++.

Transmission des objets en arguments d'une méthode :

Une méthode reçoit toujours implicitement en argument l'adresse de l'objet de sa classe qui l'a appelée.

Ainsi, pour : `a.affiche()` ;

L'adresse de `a` est transmise en argument à la fonction `affiche`. Dans la fonction, le pointeur sur l'objet qui l'a appelée est désigné par le mot clé **this**

Exemple pour la classe `point` déjà définie, les coordonnées de l'objet qui fera l'appel sont:

`x` et `y` ou `this->x` et `this->y`

Exemple d'utilisation pour la classe `point`

Refaire l'exercice Exercice 1 :

Créer une classe vect3d pour représenter un vecteur dans l'espace caractérisés par ses coordonnées x, y et z :

Et comporte un constructeur et une méthode d'affichage

Ajouter une méthode d'instance permettant de tester si deux vecteurs coïncident

Écrire un programme principal de test qui crée deux vecteurs les affiche et teste leurs coïncidence

Exercice2

- Créer une classe **Employe** permettant de représenter un employé caractérisé par des attributs privés qui sont : nom, prénom et salaire. Le nom et le prénom seront représentés sous forme de **chaînes statiques**.
- La classe comporte le constructeur sans arguments et qui permet de lire au clavier les différentes valeurs des attributs,
- le destructeur si nécessaire et
- une méthode affiche() pour afficher les différents attributs
- *Une méthode qui parmi deux employés, affiche l'employé ayant le salaire le plus grand*
- *Ecrire par ailleurs, un petit programme (main) de test*

Exercice2

- Créer une classe **Employe** permettant de représenter un employé caractérisé par des attributs privés qui sont : nom, prénom et salaire. Le nom et le prénom seront représentés sous forme de **chaînes dynamiques(char*)**..
- La classe comporte le constructeur sans arguments et qui permet de lire au clavier les différentes valeurs des attributs,
- le destructeur si nécessaire et
- une méthode affiche() pour afficher les différents attributs
- *Une méthode qui parmi deux employés, affiche l'employé ayant le salaire le plus grand*
- *Ecrire par ailleurs, un petit programme (main) de test*

Méthodes statiques

Lorsqu'une méthode a une action indépendante d'un objet quelconque de sa classe, on peut la déclarer avec l'attribut *static* (c'est-à-dire sont alloués de façon permanente en mémoire comme les variables globales).

Dans ce cas, une telle fonction peut être appelée sans mentionner d'objet particulier Selon la syntaxe suivante :

Appel:

Nom_classe :: nom_fonction(arguments) ;

Exercice:

Réaliser une classe nommée entier dont les membres données sont :

L'entier

Le compteur d'objets construits (variable statique entière)

Les fonctions membres sont :

Le constructeur, le destructeur, une fonction d'affichage et une fonction statique qui fournit le nombre d'objets construits.

- Réaliser une classe employe caractérisée par un nom, prenom, salaire et code(entier)
- Nous voulons que le code soit crée automatiquement par ordre de création des employés, 1, 2,

Fonctions Amies

Déclaration d'amitié

Il peut arriver qu'une fonction n'appartient pas à une classe ait besoin d'accéder à des données de cette classe. Une façon de procéder serait de déclarer ces données publiques, mais cela est contradictoire avec la notion de classe, dans la mesure où l'un des intérêts de l'utilisation des classes est justement de protéger les données.

La solution adoptée par C++ consiste à permettre au concepteur d'une classe de déclarer que certaines fonctions extérieures à la classe sont des amies de cette classe et sont donc autorisées à ce titre à accéder aux données privées de la classe.

Fonction externe amie d'une classe

La déclaration d'amitié est introduite dans la classe par :

friend type nom-fonction(arguments) ;

La fonction est une fonction externe standard.

Exemple :

Soit une classe point qui comporte comme membres donnés les coordonnées du point dans le plan, et comme membres fonctions le constructeur et une fonction affiche..

Ajouter une fonction externe amie de la classe point, qui **reçoit deux arguments de type point et** renvoie un boolean qui vaut true si les deux points coïncident et false si non.

Ecrire un programme principal d'essai

Remarque: La fonction **main** ne doit pas déclarer la fonction externe coïncide car celle-ci est déclarée dans la classe point.

Constructeur de recopie :

Toute classe dispose d'un constructeur de copie par défaut, similaire à l'affectation par défaut: permet de recopier les champs de l'objet à recopier un à un dans les champs de l'objet à instancier.

Toutefois, ce constructeur par défaut ne suffira pas toujours, et le programmeur devra parfois en fournir un explicitement.

Syntaxe de déclaration : *nom_classe* (**const** *nom_classe*&) ;

Si l'attribut `const` est omis, il n'y a pas d'erreur, mais il est préférable de l'utiliser, puisque le constructeur de recopie ne doit pas modifier l'objet qui lui est passé en argument par adresse.

Situations dans lesquelles le constructeur de copie est automatiquement appelé

1) Création d'un nouvel objet à partir d'un objet existant

```
Point A(1,2);
```

```
point B(A)
```

2) Transmission d'un Objet par valeur à une fonction:

```
void f(point){...}
```

3) Retour par valeur d'une fonction

```
point f() { point x,
```

```
...
```

```
return x;
```

```
}
```

Surdéfinition des Opérateurs

- En C++, on peut attribuer un nouveau rôle à un opérateur standard quand celui-ci s'applique à un objet non standard.
- Pour cela on associe à cet opérateur une fonction qui décrit l'action spécifique à réaliser lorsque l'opérateur s'applique à cet objet on dit qu'on a surdéfini cet opérateur.
- La fonction associée à l'opérateur peut être soit une fonction indépendante, amie de la classe qui définit l'objet auquel s'applique l'opérateur, soit une fonction membre de la classe.

- Le nom de la fonction est le symbole standard, précédé du mot clé **operator**.

Exemple :

Redéfinition de l'opérateur + :

la fonction est déclaré par :

type operator + (arguments) ;

❑ ***Surdéfinition par une fonction amie indépendante***

Soit un opérateur existant noté op , si op est un opérateur binaire la notation :

$a \text{ op } b$ est équivalent operator $op(a,b)$

Exemple: $a+b$ est equivalent operator $+(a,b)$

❑ ***Surdéfinition par une fonction membre***

La même notation $a \text{ op } b$ est équivalent:
 $a.operator \text{ op } (b)$

Exemple: $a+b$ est equivalent $a.operator \text{ } +(b)$

Exercice:

Utiliser l'opérateur $+$ sur la classe point pour définir un point (comme somme de deux points) dont l'abscisse sera la somme des abscisses et l'ordonnée, la somme des ordonnées.

Réaliser la surdéfinition de $+$ de deux façons :

- ✓ surdéfinition par une fonction amie
- ✓ surdéfinition par une fonction membre

la classe point a pour fonction membres le constructeur et une fonction affiche.

Règles concernant la surdéfinition des opérateurs

- Seuls les opérateurs standards peuvent être surdéfinis
- Tous les opérateurs peuvent être surdéfinis sauf l'opérateur `·`
- les règles de priorité et d'associativité sont les mêmes que pour les opérateurs standards
- Les opérateurs **new**, **delete**, **[]**, **()**, **->** ne peuvent pas être surdéfinis par des fonctions externes, mais uniquement par des fonctions membres.

Opérateur d'indexation []

- Il n'est pas possible d'accéder directement depuis une fonction utilisatrice à un élément d'un tableau déclaré dans une classe, car les éléments du tableau sont privés à la classe et la fonction utilisatrice ne peut pas accéder directement.
- fonction membre de la classe, pour surdéfinir l'opérateur d'indexation [].
- La fonction de surdéfinition doit recevoir en argument le rang de l'élément. Elle renvoie sa valeur.
- Si l'on veut pouvoir faire figurer l'élément à gauche d'un signe=, il faut utiliser la transmission par référence pour l'argument de retour.
- On ne peut pas ici avoir recours à une fonction externe avec déclaration d'amitié.

Exemple:

```
class Tableau{  
    int * a ;  
    int nmax ;  
    public :  
    ...};
```

- On considère une classe vecteur d'entiers dont les membres données sont le nombre d'éléments du vecteur et un pointeur d'entiers.
- On introduit dans cette classe une fonction de surdéfinition de `[]`. La classe peut être déclarée de façon suivante :

```
class vecteur {  
    int * a ;  
    int nmax ;  
    public :  
        int & operator [ ] (int) ; //fonction de surdéfinition de  
        l'indexation  
        ... } ;
```

On peut définir la fonction d'indexation par :

```
int & vecteur :: operator [ ] (int i) {  
    return *(a+i) ; }
```

Exercice :

Donner un programme complet pour la classe vecteur ci- dessus, en la dotant d'un constructeur, d'un destructeur, d'une fonction de remplissage, d'une fonction d'affichage et de la fonction de surdéfinition de [].

La fonction utilisatrice double les éléments de rang impair.

Surdéfinition de l'affectation

- Indispensable de la surdéfinir lorsqu'elle concerne des objets alloués dynamiquement
- l'opération d'affectation standard consiste simplement à recopier les membres donnés de l'objet figurant à droite du signe égal dans les membres donnés de la variable réceptrice. Elle ne recopie pas les contenus des zones mémoires allouées dynamiquement.

Considérant:

```
class vecteur {  
    int * a ;  
    int nmax ;  
public :  
    vecteur (int k) {  
        a=new int[nmax=k] ;}  
        ...  
    } ;
```

Si l'on déclare dans une fonction utilisatrice

vecteur x(n) ;

vecteur y(m) ;

y=x ;

La valeur m de y est changée en n, et celle du membre donnée **a** est changée en l'adresse du tableau alloué dynamiquement pour x.

ceci a pour effet de conserver en mémoire l'emplacement créé dynamiquement par le constructeur lors de la déclaration **vecteur y(m)**, et qui n'est plus référencé par y. cet emplacement risque de n'être jamais libéré.

Pour éviter ce genre de situation, il faut surdéfinir l'affectation en prenant soin :

- de libérer l'espace alloué par le constructeur pour le vecteur y
- de créer une nouvelle zone mémoire de la taille du vecteur x, dans laquelle on va recopier les valeurs des éléments de x.
- d'affecter aux membres données du vecteur y les valeurs qu'ils doivent avoir, c'est à dire x.nmax pour le nombre d'éléments et l'adresse de la nouvelle zone créée pour y pour le membre y.a
- De ne rien faire s'il s'agit d'une affectation de l'objet à lui même.

La fonction de surdéfinition de l'affectation d'un vecteur à un autre s'appellera ***operator =***

Elle recevra en argument explicite le vecteur à droite du signe =. Ici, nous avons choisi de passer cet argument par adresse, pour des raisons de performances, en précisant l'attribut *const* puisqu'il ne doit pas être modifié. La fonction reçoit implicitement en argument la variable réceptrice

Définir cette fonction?

```
Vecteur& vecteur :: operator = (const vecteur &u) {  
    if (this !=&u) {  
        delete [ ] a ;  
        nmax=u.nmax;  
        a=new int[nmax];  
        for (int i=0; i<nmax; i++)  
            *(a+i) = *(u.a+i) ; }  
        else cout << " on ne fait rien \n ";  
        return *this; }
```

Ici ***this*** est l'adresse de l'opérande de gauche (passé implicitement en argument).

Exercice : (à faire en TP)

- Réaliser une classe vecteur d'entiers dont les membres données sont le nombre d'éléments du vecteur et un pointeur d'entiers.
- avec toujours pour fonctions membres
 - -le constructeur qui reçoit la taille du vecteur qui permet de créer et remplir ce vecteur au clavier
 - -Le destructeur
- (Afficher les adresses des différents objets créés et détruits)
 - -une fonction de surdéfinition de []
 - -une fonction d'affichage
 - -la fonction de surdéfinition de l'affectation
- Ecrire un programme principal de test.

Héritage Simple

1 Création d'une classe dérivée à partir d'une classe de base:

syntaxe de définition d'une nouvelle classe B dérive de A :

```
class B :public A  
{// définition des membres supplémentaires (données ou  
fonctions)  
  
...  
};
```

Exemple 1:

à partir de la classe *point* que nous avons déjà utilisée (classe de base), nous voulons créer une autre classe (classe dérivée) en ajoutant un symbole identifiant le point, qui sera une suite de un ou plusieurs caractères. Nous appelons la classe dérivée *pointcar*.

La déclaration des membres donnés de la classe dérivée s'écrit :

```
class pointcar : public point {  
    char * sy ;  
    Public :  
    // ...    fonctions membres propres à la classe dérivée  
};
```

La classe dérivée *pointcar* a accès aux membres publics de la classe de base *point*, mais pas à ses membres privés.

Exemple 2:

à partir de la classe *point* que nous avons déjà utilisée (classe de base),

Créer une classe *cercle* (classe dérivée) en ajoutant un rayon.

Constructeur d'un objet d'une classe dérivée

La classe dérivée doit comporter un constructeur qui initialise les membres données n'existant pas dans la classe de base.

se déclare par :

pointcar (float , float , char *);

et se définit par:

```
pointcar: : pointcar ( float x=0., float  
    y=0., char * c=" " ):point(x,y){  
    ....  
}
```

Appel d'une fonction de base depuis une fonction membre

Une fonction de la classe dérivée peut appeler n'importe quelle fonction publique de la classe de base, On utilise l'opérateur de résolution de portée ::

Nom_classe::nom_fonction(arguments)

Exemple: On prévoit dans la classe dérivée *pointcar* une fonction d'affichage appelée *affichec()* qui appelle la fonction *affiche()* de la classe *point* pour afficher le nom et les coordonnées d'un point

```
void pointcar ::affichec ( ) {  
    point :: affiche ( ) ;  
    cout <<" identificateur : "<< sy << endl ;}
```

Programme principal

en supposant que la classe de base est définie dans le fichier *point-base.h* et la classe dérivée dans *carpoint.h*

```
# include " point-base.h"
```

```
# include " carpoint.h"
```

```
// fonction utilisatrice
```

```
main ( ) {
```

```
pointcar p(1.5, 2, "A" ) ;p. affichec ( ) ;}
```

Le résultat est le suivant :

passage dans constructeur de point

passage dans constructeur de pointcar

coordonnees : 1.5 2

identificateur : A

Redéfinition d'une fonction de la classe de base

Une classe dérivée peut avoir une fonction membre de même nom que la classe de base et redéfinir celle-ci

Toujours La fonction de la classe dérivée fera appel à la fonction de la classe de base en utilisant l'opérateur de résolution de portée :: et pourra compléter l'action de celle-ci.

Ainsi, au lieu de créer une fonction *affichec* membre de la classe dérivée, il est préférable de redéfinir avec le même nom la fonction *affiche* de la classe *point*.

Exercice:

Ecrire un programme complet de la classe point et pointcar avec les fonctions membres constructeur, affiche(). Ecrire un programme principal de test

Exercice2:

Ecrire un programme complet de la classe
point et cercle avec les méthodes
Constructeur, redéfinition de affiche().
Ecrire un programme principal de test

Contrôle des accès à la classe de base

- a. Statut protégé aux membres de la classe de base

En plus des statuts *private* et *public*, le C++ a introduit le statut *protected*.

Exemple :

```
class point{
```

```
Protected :
```

```
float abs,float ord ;
```

```
....};
```

Les membres qui ont ce statut sont accessibles, aux fonctions membres de la classe et aux fonctions amies, et de plus, aux éventuelles **classes dérivées**, mais pas aux utilisateurs des classes dérivées.

Exercice 2 :

En utilisant ce statut, refaire l'exercice 2 précédent

Conversion

Affectation d'un objet dérivé à un objet de base

On peut affecter un objet d'une classe dérivée à un objet de base, mais pas l'inverse.

Exercice

Créer un *point* et un *pointcar*, et affecter le *pointcar* au *point*, ce qui est licite. Par contre une affectation d'un *point* à un *pointcar* conduit à une erreur de compilation. Dans le programme principal, créer un point *pa*(0,0) et un *pointcar*

p(1.5, 2, "X1"). Que devient *pa* après l'affectation *pa*=*p*.

Le programme conduit au résultat suivant :

coordonnees : 1.5 2

Affectation par pointeur

- On peut affecter un **pointeur** sur un objet dérivé à un pointeur sur un objet de base.
- L'inverse est possible à condition d'utiliser l'opérateur de cast.
- Mais Attention, si par la suite on applique une fonction à l'objet pointé par ce pointeur, c'est la fonction définie dans la classe dérivée qui s'applique, comme le montre l'exemple ci-dessous :

Exemple

Comme le montre le programme principal ci-dessous concernant une classe pointcar qui dérive de la classe point :

// Fonction utilisatrice

```
main ( ) {  
pointcar a( 1.5, 2., " A " ) ;  
point x(-0.5,3.) ;  
pointcar * b=&a ;  
point * y=&x ;  
y=b ;b->affiche ( ) ;y->affiche ( ) ;  
point z(1.,2.7) ;point *u=&z ;  
b= (pointcar *)u ;u->affiche ( ) ;b->affiche ( ) ;}  
le résultat obtenu est le suivant :
```

passage dans constructeur de point

passage dans constructeur de pointcar

passage dans constructeur de point

coordonnees : 1.5 2

identificateur : A

coordonnees : 1.5 2

passage dans constructeur de point

coordonnees : 1 2.7

coordonnees : 1 2.7

identificateur : ??

Appel d'une fonction de base par un objet dérivé

- Une fonction de base peut être appelée par un objet d'une classe dérivée. En effet, il y a conversion implicite de l'objet dérivé dans le type de base.

Les fonctions virtuelles

- Typage statique des objets

Les règles de compatibilité entre une classe de base et une classe dérivée permettent d'affecter à un pointeur sur un objet de base la valeur d'un pointeur sur un objet dérivée.

Toutefois, **par défaut, le type des objets pointés en C++, est déterminé à la compilation.** C'est ce qu'on appelle le typage statique.

Par exemple, avec :

```
class A  
{ ...  
  public :  
  void fct(...);  
  ...  
};
```

```
class B: public A  
{ ...  
  public :  
  void fct(...);  
  ...  
};
```

```
A *pta;  
B *ptb;  
pta=ptb est autorisée.
```

Néanmoins: pta->fct(...) appelle toujours la fonction fct de la classe A.
(pta est toujours considéré comme un pointeur sur A).

Lorsqu'une fonction est appelée le type d'objet est alors considéré être celui qui est déclaré dans la fonction utilisatrice, et non pas son type effectif .on dit que le type des objets pointés est déterminé à la compilation.

Il serait intéressant de pouvoir déterminer le type d'un objet à l'exécution. Ceci permettrait d'affecter à un objet pointé son type effectif et non pas son type déclaré.

Ceci est possible en C++ grâce au mécanisme des fonctions virtuelles. On parle alors du typage dynamique (ou polymorphisme).

Typage dynamique et fonctions virtuelles

On souhaite que, lorsqu'une fonction d'une classe est appelée, la fonction s'applique à l'objet effectif, et non pas à l'objet déterminé à la compilation. C'est ce que l'on appelle **le polymorphisme**.

Pour cela, il suffit de déclarer la fonction virtuelle dans la classe de base.

Par exemple, si l'on a :

```
class A
{ ...
  public :
  virtual void fct(...);
  ...
};
```

```
class B: public A
{ ...
  public :
  void fct(...);
  ...
};
```

A *pta;

B *ptb;

Ainsi, si pta=ptb. pta->fct(...) appelle la fonction fct de la classe B.

- **Remarque :**

Pour que la liaison dynamique puisse se faire, il faut que la fonction soit appelée par un pointeur et non directement par l'objet lui-même.

Reprenons le même exemple **en donnant l'attribut virtual à la fonction affiche()** dans la classe de base **point**.

```
# include < iostream.h>
class point {
float abs ; float ord ;
public :
point ( float x=0., float y=0.){
abs=x ; ord=y ;}
void virtual affiche ( ) {
cout <<" coordonnees : " <<abs<< " " <<ord << endl ; } } ;

class pointcar : public point {
char * sy ;
public:
pointcar ( float x=0., float y=0., char* c=" *" ):
point(x,y){
cout <<" passage dans constructeur de pointcar"<<endl;
sy= c ;}
void affiche() ;
} ;
void pointcar ::affiche(){
point ::affiche() ;
cout<<identificateur<<sy<<endl ;
}
```

// fonction utilisatrice

```
main ( ) {  
pointcar a(1.5, 2, "A") ;  
point x(-0.5,3.) ;  
pointcar *b=&a ;  
point * y=&x ;  
y=b; b->affiche ( ); y->affiche ( );  
}
```

Le résultat est le suivant :

Coordonnées : 1.5 2

Identificateur : A

Coordonnées : 1.5 2

Identificateur : A

Ici, le type effectif de b est point avant affectation, son type devient pointcar suite à l'affectation y=b.

HERITAGE MULTIPLE

Principe : une classe peut dériver de plusieurs classes.

Dérivation à partir de plusieurs classes

- Déclaration d'une classe dérivée de plusieurs classes

Soient deux classes A et B déclarées par :

```
class A {  
    ... };
```

```
class B {  
    ... };
```

Pour définir une nouvelle classe C qui dérive de A et B, on la déclare par :

```
class C : public A, public B{  
    ... } ;
```

Exemple:

Soit les classes ***Salaire*** et ***Identificateur*** suivantes :

```
class Salaire {  
double S ;  
public :  
Salaire(double) ;  
... } ;
```

```
class Identificateur {  
char* Nom  
public :  
Identificateur(char*) ;  
... } ;
```

Nous pouvons créer une classe **employé** caractérisé par le nom de l'employé et son salaire en la dérivant à partir des deux classes précédentes.

Syntaxe de déclaration :

```
class Employé : public Salaire, public Identificateur{  
... } ;
```

Constructeur pour la classe dérivée

Comme pour les dérivations simples, les constructeurs des classes de base sont appelés dans l'ordre où ils sont déclarés dans la classe dérivée.

Le constructeur de la *classe dérivée Employé* déclarée ci-dessus peut s'écrire :

```
Employé : : Employé( double a=0, char * c= " ") : Salaire(a),  
    Identificateur(c) {  
    cout << " passage dans constructeur de Employé<<endl " ;}
```

Redéfinition de fonctions définies dans les classes de base

Exemple : Redéfinition de la fonction affiche dans la classe employé : celle-ci n'a rien d'autre à faire que d'appeler successivement les fonctions affiche des classes Salaire et Identificateur. On peut l'écrire :

```
void Employé :: affiche ( ) {  
    Salaire :: affiche ( ) ; Identificateur :: affiche( ) ;}
```

Exercice

Donnez programme complet de définition et d'utilisation de la classe Employé à partir des classes *Salaire* et *Identificateur* (avec les constructeurs et les fonctions d'affichage)

Désignation des membres données

Si des membres donnés des classes de base accessibles, portent le même nom dans plusieurs classe de base, cela ne pose pas de problème. Il suffit d'utiliser l'opérateur de résolution de portée :: pour spécifier la classe dans laquelle il faut prendre l'objet. Ainsi :

A :: x désigne un objet x membre de la classe A

B :: x désigne un objet x membre de la classe B

Exemple:

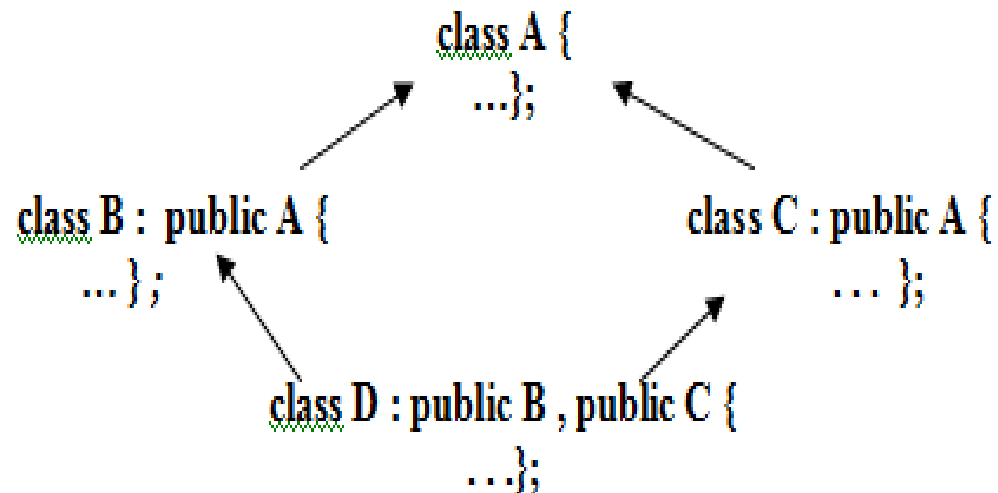
Donnons aux membres de la classe Salaire et de la classe Identificateur le statut *protected* et introduisez dans la classe Employé une fonction *coïncide* qui renvoie un *bool* valant *true* si 2 employés possèdent le même nom et même salaire, et *false* sinon. Nous supposons que le nom du membre donné de la classe Salaire est x , et le nom du membre donnée de Identificateur est aussi x. Ecrire le programme complet.

Classes Virtuelles

Problème de la duplication de données

Il peut arriver qu'une classe dérive de plusieurs classes qui dérivent elles-mêmes d'une même classe de base. Dans ce cas, les membres de la classe de base vont apparaître plusieurs fois dans la dernière classe obtenue par dérivation.

Exemple:



On dit que D hérite deux fois de A, les membres donnés seront dupliqués dans tous les objets de type D.

Déclaration de classes:

```
class B : public A  
    { ... } ;
```

```
class C : public A  
    { ... } ;
```

```
class D : public B , public C { ... } ;
```

Lorsqu'on crée un objet de type D, Les constructeurs sont appelés dans l'ordre suivant:

- constructeur de A avec la liste d'arguments passés par B
- constructeur de B avec la liste d'arguments passés par D
- constructeur de A avec la liste d'arguments passés par C
- constructeur de C avec la liste d'arguments passés par D
- constructeur de D

On voit qu'il y a bien construction de deux objets de type A

Solution: Introduire classe virtuelle

Pour les méthodes, cela ne présente pas d'inconvénient majeur, car les fonctions sont générées à la compilation et leur code est introduit une seule fois à l'édition de liens. Il n'y a donc pas de duplication. Mais les attributs, eux, sont réellement dupliqués.

Le C++ permet de n'incorporer qu'une seule fois les données. Il suffit pour cela de déclarer dans les classes *B* et *C* que la classe *A* est virtuelle. On spécifie tout simplement l'attribut ***virtual*** dans les déclarations des classes *B* et *C*. On écrira:

```
class B : public virtual A
{ ... };
```

```
class C : public virtual A
{ ... };
```

```
class D : public B , public C { ... };
```

L'attribut *virtual* n'a pas d'effet sur les classes *B* et *C*, mais uniquement sur les descendants éventuels de *B* et *C*.

Dans ce cas de classe virtuelle il ya des changements au niveau **d'appel de constructeurs**

- On évite d'appeler deux fois le constructeur de A, d'abord depuis B, puis depuis C.
- Le C++ traite les choses de la façon suivante:
les classes où une classe est déclarée virtuelle n'appellent pas le constructeur de la classe virtuelle.

Les constructeurs seront déclarés ainsi :

A: :A(arg) {...}	pour la classe A
B::B(arg) {...}	pour la classe B
C::C(arg) {...}	pour la classe C
D::D(arg) : A(arg), B(arg), C(arg) {...}	pour la classe D

Ils sont appelés dans l'ordre suivant:

- constructeur de A avec les arguments passés par D
- constructeur de B avec les arguments passés par D
- constructeur de C avec les arguments passés par D
- constructeur de D

Exercice:

- Ecrire une classe étudiant caractérisée des attributs nom, prenom code et d'une moyenne, un constructeur,
- une classe Enseignant caractérisée par le nom, prenom, le nombre d'heure enseignées et un taux horaire. Et comporte un constructeur
- Certains étudiants peuvent effectuer des enseignements (vacations). En utilisant l'héritage multiple, Ecrire, une classe etudiantVacataire héritant de Enseignant et etudiant et comporte un constructeur.
- Ecrire un programme principal de test

Exercice héritage simple et polymorphisme

- Classes

point → Cercle → Cylindre

Avec les fonctions d'affichage polymorphes
et constructeurs

Partie2

Programmation Orientée Objet avec: JAVA

- **Présentation de Java**
- **Éléments de base de programmation en Java**
- **Les classes et les Objets**
- **Les tableaux et les chaines de caractères**
- **Héritages et polymorphisme**
- **Gestion des exceptions**
- **Les interfaces graphiques**

1. Caractéristiques du Langage JAVA

- **Simple**
 - Syntaxe proche de celle de C/C++
 - pas de pointeurs
 - la gestion de la mémoire est effectuée automatiquement
 - ...

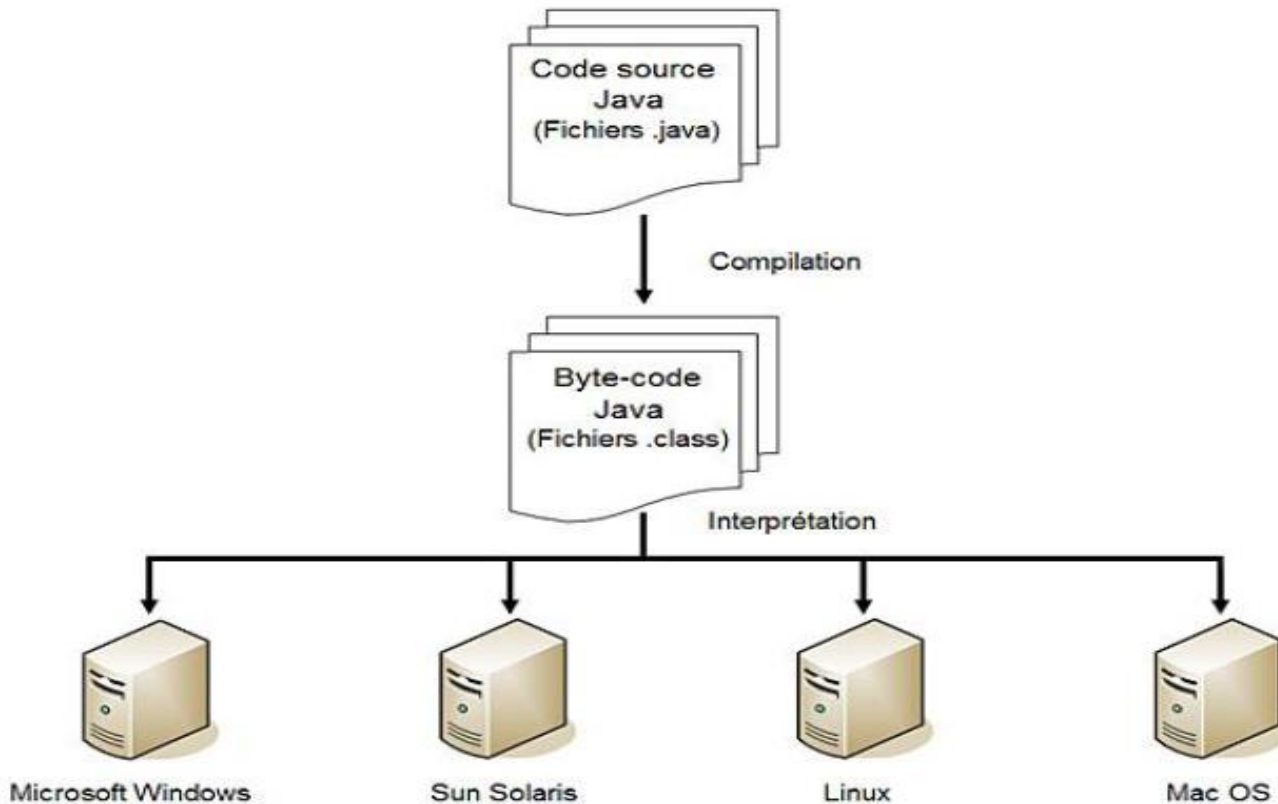
□ **Portabilité:**

Java est indépendant de l'architecture

*Portable sur plusieurs systèmes d'exploitation tels
que UNIX, Windows, Mac OS ou GNU/Linux*

« Ecrire une fois, exécuter partout »

Langage de programmation interprété et compilé



L'interprète est une machine virtuelle plus connue sous le nom de «JVM ».

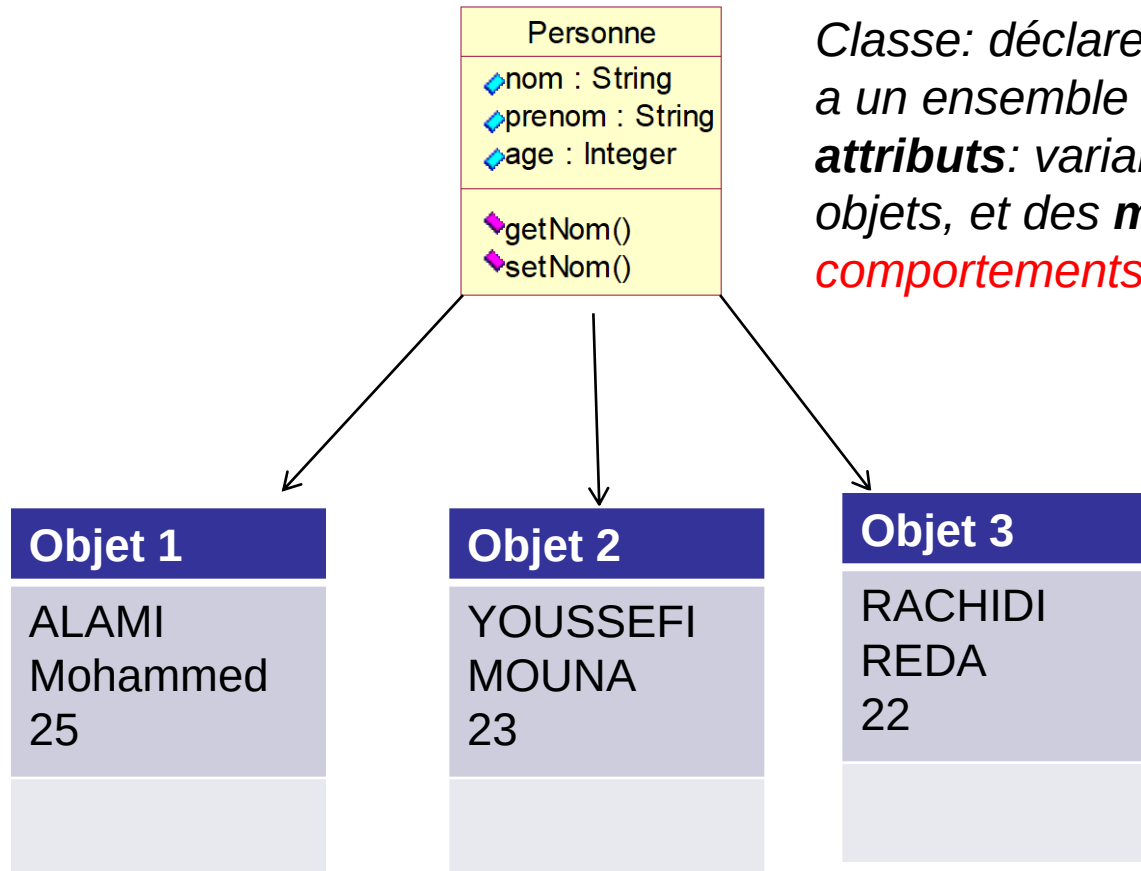
- **Orienté objet**

Java est un *langage purement Orientée Objet*:

– Les grandes idées reprises sont :

- ✓ Encapsulation (les données sont protégées à l'intérieur de chaque objet Les données (publiques, secrètes)
- ✓ Héritage
- ✓ Polymorphisme
- ✓

Concept de classe et Objet



*Classe: déclare des propriétés communes a un ensemble d'objets. Elle définit des **attributs**: variables représentant l'état des objets, et des **méthodes** définissant leurs comportements.*

*Objet=Instance de classe
Dont les attributs ont des valeurs*

2. Environnement de JAVA

Outil de développement : le JDK

- JDK signifie Java Development Kit (Kit de développement Java).
- Il contient essentiellement:
 - API java les classes de base(plusieurs centaines),
 - le compilateur : javac
 - la JVM (machine virtuelle)

Notion de PAQUETAGES (les packages)

- un paquetage est un dossier qui regroupe plusieurs classes qui collaborent entre elles dans une application.
- Permet de définir un certain niveau de protection.

Utilisation des classes d'un paquetage

- *import* nomDuPackage; ou *import* nomDuPackage.* ;
- *import* nomduPackage.nomDeLaclasse ;
- *import* nomduPackage.n* ;

L'API de Java

API (Application and Programming Interface) : Ensemble de bibliothèques et de classes

Ces classes sont regroupées, par catégories, en paquetages (ou "packages").

- ***Les principaux paquetages***

- java.util : structures de données classiques
- java.io : entrées / sorties
- java.lang : chaînes de caractères, interaction avec l'OS, threads
- java.applet : les applets sur le web
- java.awt : interfaces graphiques, images et dessins
- javax.swing : package récent proposant des composants « légers » pour la création d'interfaces graphiques
- java.net : sockets,
- ...

Exécution d'un programme Java:

Deux façons:

- Passer par la fenêtre invitée de commandes
Dos:

Il suffit d'avoir JDK et un éditeur de texte(bloc notes...)

- Environnement à interface graphique
(Eclipse...)

Passer par la fenêtre invitée de commandes Dos:

JDK premier outil de développement Java, sans utiliser d'interface utilisateur graphique, passer par la fenêtre invitée de commandes Dos pour pouvoir compiler, exécuter,

Le Kit JDK ne comporte pas d'éditeur pour la saisie du code source: on utilise n'importe quel éditeur de texte (Bloc-notes, commande edit de Ms-Dos, wordpad...)

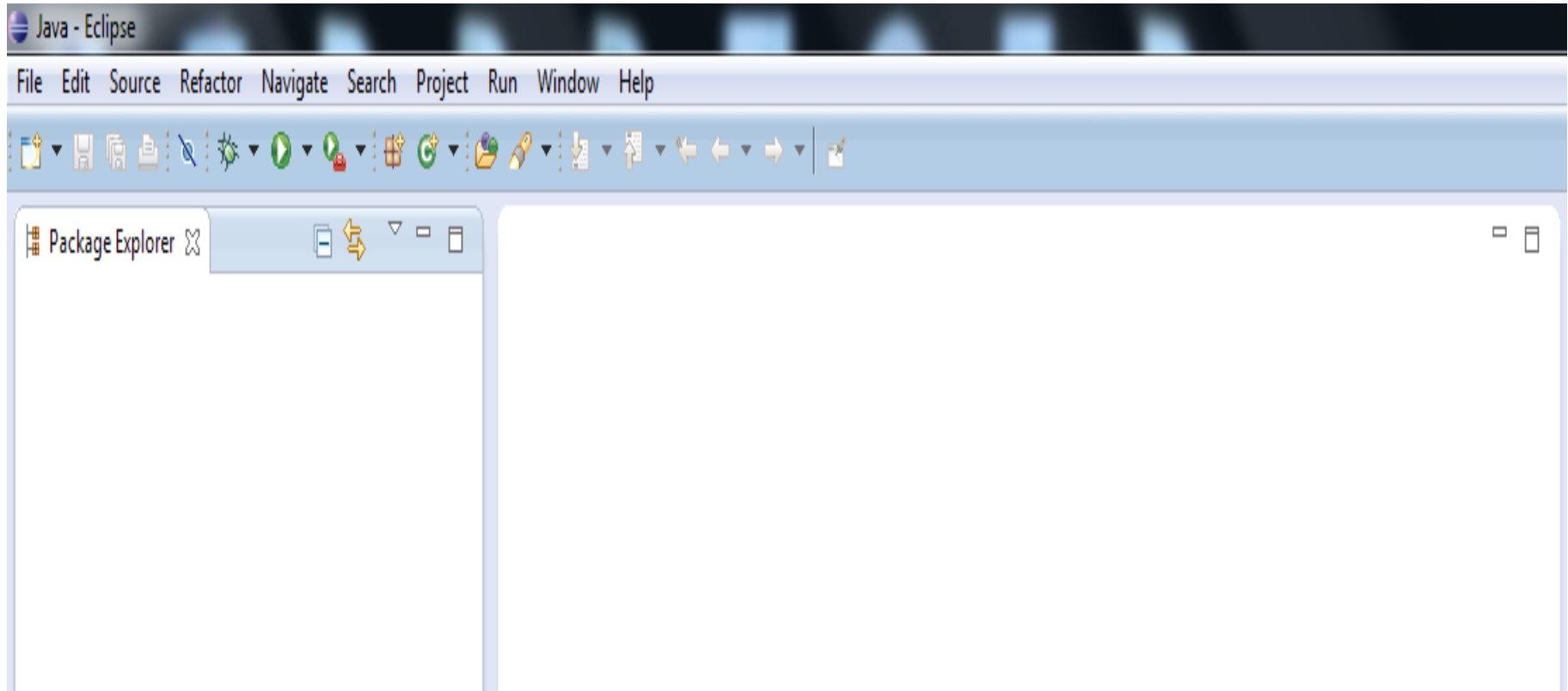
Taper, compiler et exécuter en JDK

- ***Code source :*** *lancer l'éditeur de texte de votre choix , saisir puis enregistrer le fichier source avec le même nom de la classe et avec une **extension .java***
- ***La compilation :*** ***javac NomFichier.java***
- ***L'exécution:*** ***java NomFichier***

Remarque: *Le programme java est exécuté sur tout ordinateur pour lequel une machine virtuelle est disponible.*

Utilisation d'environnement à interface graphique

Eclipse, Jbuilder,...



Structure générale d'un programme Java:

Un programme Java est composé au moins par une classe:

```
public class test1{  
    public static void main(String arg[]){  
        ... code du programme  
    }// main  
}// class
```

La fonction main, appelée aussi méthode est exécutée la première lors de l'exécution d'un programme Java. Elle doit avoir obligatoirement la signature précédente :

```
public static void main (String arg[]){  
ou  
public static void main(String[] arg){
```

- Premier exemple de programme en Java

```
public class PremierProg {  
    public static void main (String[ ] argv) {  
        System.out.println(" Mon Premier Programme");  
    }  
}
```

« System.out.println("Hello World") » : revient à la ligne

« System.out.print("Hello World") » : ne revient pas à la ligne

Ou `java.lang.System.out.println("Hello World");`

```
public final class System extends Object{
```

```
....  
static PrintStream out ;
```

```
.....
```

```
public class PrintStream extends FilterOutputStream implements  
Appendable, Closeable...
```

`Println` c'est une méthode de la classe **PrintStream**.

Chapitre1:

Éléments de base de programmation en Java

Syntaxe du langage Java

Règles générales d'écriture du code java

c- Les commentaires

Ils ne sont pas pris en compte par le compilateur donc ils ne sont pas inclus dans le pseudo code. Ils ne se terminent pas par un ;.

Il existe trois type de commentaire en Java

Type de commentaires	Exemple
commentaire abrégé	// commentaire sur une seule ligne int N=1; // déclaration du compteur
commentaire multiligne	/* commentaires ligne 1 commentaires ligne 2 */

- Les types de données prédéfinis

Type	Désignation	Longueur	Valeurs	Commentaires
boolean	valeur logique : true ou false	8 bits	True ou false	pas de conversion possible vers un autre type
byte	octet signé	8 bits	-128 à 127	
short	entier court signé	16 bits	-32768 à 32767	
char	caractère Unicode	16 bits	\u0000 à \uFFFF	entouré de cotes simples dans un programme Java
int	entier signé	32 bits	-2147483648 à 2147483647	
float	virgule flottante simple précision (IEEE754)	32 bits	1.401e-045 à 3.40282e+038	
double	virgule flottante double précision (IEEE754)	64 bits	2.22507e-308 à 1.79769e+308	
long	entier long	64 bits	-9223372036854775808 à 9223372036854775807	

Déclaration des données

Déclaration des constantes

La syntaxe de déclaration d'une constante est la suivante :

final type nom=valeur; //définit constante nom=valeur

exemple :

```
final float tva=20;
```

```
tva = tva -2 ; // erreur : tva a été déclarée final
```

Fonction d'entrées / sorties standards

a- Ecriture sur écran

La syntaxe de l'instruction d'écriture sur l'écran est la suivante :

System.out.println(expression) ou *System.err.println(expression)*

où *expression* est tout type de donnée qui puisse être converti en chaîne de caractères pour être affiché à l'écran.

b- Lecture de données tapées au clavier

- Java dispose de méthode d'affichage d'information dans la fenêtre de console,

Rien n'est prévu pour la lecture au clavier.

Pour faciliter la résolution, notre environnement de travail **Eclipse** dispose d'une **classe Scanner** qui nous facilite la tâche.

```
import java.util.Scanner;
public class Hello
{
    public static void main(String args[])
    {
        Scanner sc= new Scanner(System.in); //in champs de type inputstream
        System.out.println("Veuillez saisir un entier");
        int n1 = sc.nextInt();
        System.out.println("la valeur de n="+n1);
    }
}
```

Exercice:1

Java dispose de méthode d'affichage d'information dans la fenêtre de console, rien n'est prévu pour la lecture au clavier. Bien entendu, il est toujours possible de développer soi-même une classe offrant les services de base qui sont lecteur d'un entier, d'un flottant, d'un caractère ou d'une chaîne de caractères. Pour faciliter la résolution notre environnement de travail eclipse dispose d'une classe Scanner qui nous facilite la tâche

```
import java.util.Scanner;
public class Hello
{
    public static void main(String args[])
    {
        Scanner sc= new Scanner(System.in);
        System.out.println("Veuillez saisir un
mot");
        String n = sc.nextLine();
        System.out.println("la valeur de n="+n);
        System.out.println("Veuillez saisir un
entier");
        int n1 = sc.nextInt();
```

```
System.out.println("la valeur de n="+n1);
System.out.println("Veuillez saisir un
real");
        float n2 = sc.nextFloat();
        System.out.println("la valeur de n="+n2);
        double n2 = sc.nextDouble();
        System.out.println("la valeur de n="+n3);
    }
}
```

Exercice 1: Savoir compiler et exécuter un programme java sous MSDOS sans utilisation des outils de développements graphique, (Eclipse, NetBeans, Jbuilder,...)

- Après installation du JDK, ajouter le chemin de la commande javac à la variable d'environnement PATH (Panneau de configuration\ Système et sécurité\ Système \ paramètres systèmes avancés\ variables d'environnements)
- Lancer bloc notes ou autre éditeurs de texte et taper le code suivant :

```
public class PremierProg {  
    public static void main (String[ ] argv) {  
System.out.println(" Bonjour, mon Premier Programme en ligne de commande");  
    }  
}
```

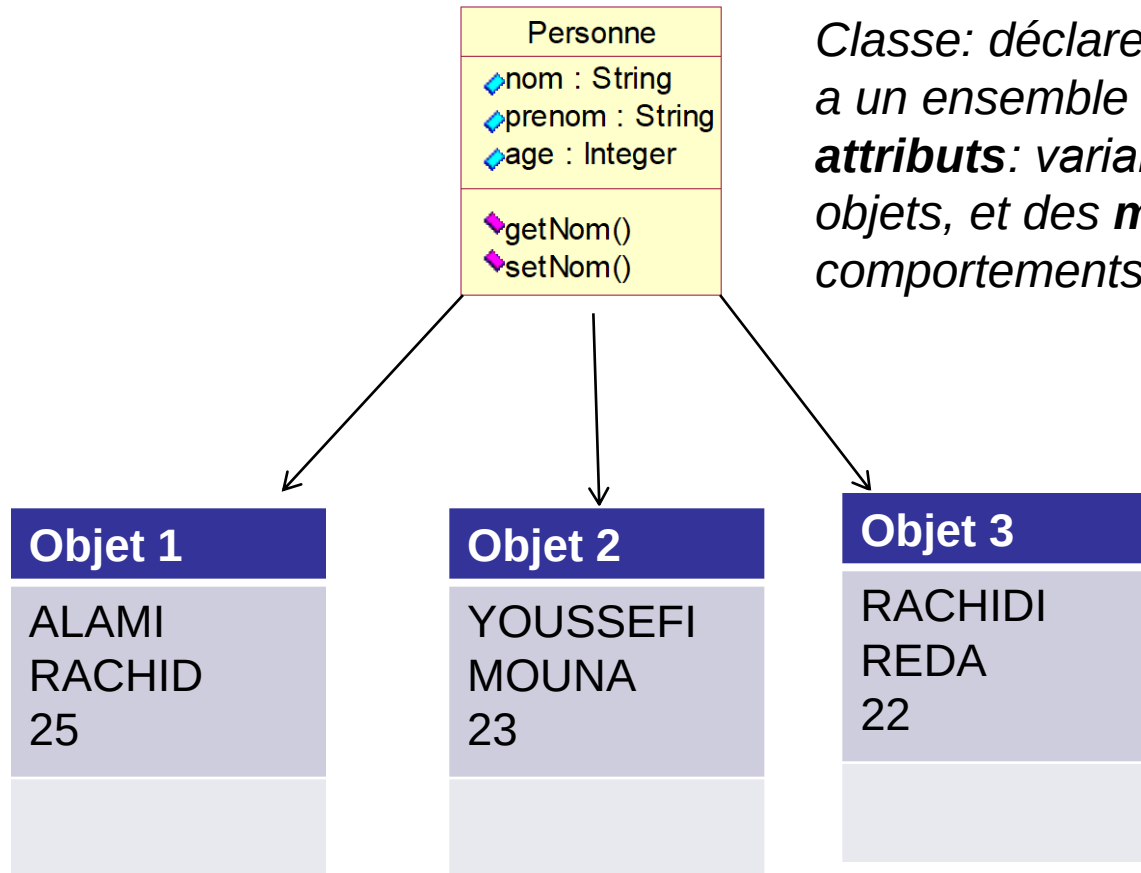
Enregistrer le code dans un fichier portant le même nom que la classe avec l'extension java(**PremierProg.java**)

Compiler le programme : javac **PremierProg.java**

Exécuter votre programme : java **PremierProg**

Chap 2. Les Classes et Les Objets

Concept de classe et Objet



*Classe: déclare des propriétés communes a un ensemble d'objets. Elle définit des **attributs**: variables représentant l'état des objets, et des **méthodes** définissant leurs comportements.*

*Objet=Instance de classe
Dont les attributs ont une valeur*

définition d'une classe

La syntaxe de définition d'une classe est la suivante :

```
Public class NomdeClasse  
{  
    définition:  
        des variables (données)  
        des méthodes  
}
```

Le mot clé **public** permet l'accès de toutes autres classes à cette classe

En son absence l'accès aux seules classes du même package

Un premier exemple

```
class point
{
    private float x;
    private float y;
    Public void affiche(){
        System.out.println("l'abscice="+x+"l'ordonné="+y);
    }
}
```

Un attribut est **private**, il est accessible uniquement depuis sa propre classe

par défaut (rien): elle n'est visible alors que de son propre paquetage

- UTILISATION DES CLASSES

Pour utiliser une classe, elle doit être **instanciée** (à partir de n'importe quel méthode). Il y aura création d'un objet.

Mais avant de créer un objet, il faut le déclarer.

- Déclaration d' objet

NomClasse Objet ;

- Cette instruction déclare un objet de la classe NomClasse mais ne le crée pas *réserve uniquement un emplacement pour une référence à l'objet.*

Exemple: Point a;

- Création d'un objet

Objet = new NomClasse() ;

La déclaration et la création d'un objet peuvent être regroupées en une seule instruction :

NomClasse objet = new NomClasse() ;

Exemple: Point a=new point();

On accède aux méthodes et aux attributs de la classe
comme suit :

NomObjet.méthode(arguments de la méthodes) ;

NomObjet.attribut ;

Exemple:

a.affiche();

**Ecrire l'exemple complet de la classe point et d'une classe
test_point contenant la méthode principale**

Quel résultats vous avez obtenu après exécution

```
class point  
{  
private float x;  
private float y;  
Public void affiche(){  
System.out.println("l'abscice="+x+"l'ordonné="+y);  
}}
```

```
class test  
{  
public static void main (String[ ] arg)  
{  
point A = new point() ;  
A.affiche() ;  
}  
}
```

1- Initialisation par défaut des champs d'un objet (constructeur par défaut)

Dès qu'un objet est créé, ses champs sont initialisés à une valeur par défaut comme suit :

Type du champ	Valeur par défaut
boolean	false
char	caractère de code nul
entier (byte, short, int, long)	0
flottant (float, double)	0.f ou 0.

La Notion de CONSTRUCTEUR

Définition

- Le constructeur est une méthode qui est appelée au moment de la création d'un objet. Il sert **à créer et à initialiser** les variables contenues dans les objets.

Le constructeur porte le même nom que sa classe. Il ne retourne pas de valeur et ne mentionne pas void au début de sa déclaration

- Toute classe peut posséder plusieurs constructeur. Si une classe ne déclare pas de constructeur, un constructeur vide est appelé automatiquement par défaut.
- un constructeur est une fonction qui est appelée par le biais de l'opérateur new.

monObjet = new Constructeur(arguments)

Exemple de constructeur pour la classe point:

Public class point

```
{
    private float x ;
    private float y ;
    public point( float abs, float ord)
    {
        x = abs ;
        y = ord ;
    }
    public void affiche()
    {
        System.out.println(" Point de coordonnées " + x+ "  " +y) ;
    }
}
```

class test

```
{
    public static void main (String[ ] arg)
    {
        point A = new point(1,1) ;// point A = new point() ; sera refusé par le compilateur
        A.affiche() ;
    }
}
```

Initialisation:

Un attribut peut être initialisé lors de sa déclaration, cette valeur sera écrasé par le constructeur

Exemple:

```
Public class point
{
    private float x=10 ;
    private float y=20 ;
    public point( float abs, float ord)
    {
        x = abs ;
        y = ord ;
    }
}
```

Afféctation d'objets

```
Point a=new point(3,5);
```

```
Point b=new point(1,4);
```

```
a=b;
```

Maintenant a et b désignent le même objet (il n'ya plus deux objets)

De même:

```
if(obj1 == obj2)
```

est vraie si deux référence obj1 et obj2 désignent le même objet

On compare les références et non pas le contenu.

C'est la **méthode d'instance** equals qui permet de comparer le contenu des attributs

Ramasse-miettes

- Il n'existe aucun opérateur permettant de détruire un objet, ça se fait automatiquement par un processus dit: ramasse miettes (garbage collector)

Modes de passage d'arguments à une méthode

Java transmet toujours les paramètres **par valeurs**

- Mais lorsqu'un objet est transmis, son nom représente sa référence, d'où l'utilisation du passage par référence pour les objets

Par conséquent, la méthode permet la modification de l'objet concerné

Exemple:

permutation de deux objets (deux points par exemples)

- Réaliser la classe point avec les méthodes constructeur explicite, d'affichage, permutation de deux points
- Écrire une classe principale de test

Auto référence: le mot clé this

Considérant l'appel d'un objet à une méthode Exemple:

```
A.affiche();
```

La méthode reçoit automatiquement la référence à l'objet qui fait l'appel (this):
objet courant

Exemple d'utilisation de this:

```
Public class point
{
    private float x ;
    private float y ;
    public point( float x, float y)
    {
        this.x = x ;
        this.y = y ;
    }
}
```

Remarque **this** : utile lors de la conception d'arguments d'une méthode d'instance

Exemple: méthode coïncide.

Exemple2:

- Réaliser la classe point avec les méthodes constructeur explicite, d'affichage, et une méthode **coincide permettant de tester si deux points coïncident**
- Écrire une classe principale de test

Surcharge de méthodes

Possibilité de définir des méthodes possédant le même nom mais dont les arguments diffèrent (**nombre ou/ et le type ou/ et ordre d'arguments**)

Possibilité de surcharge de constructeur

Quand une méthode surchargée est invoquée le compilateur sélectionne automatiquement la méthode dont **le nombre et le type des arguments** correspondent au nombre et au type des paramètres passés dans l'appel de la méthode.

Exemple de surcharge de constructeur

- Constructeurs de la classe point à 0, 1 ou 2 arguments:

```
Public class point{  
Private int x,y;  
Public point(){x=0;y=0;}  
Public point(int abs){x=abs;y=abs;}  
Public point(int abs,int ord){x=abs;y=ord;}  
//class de test:  
Public class surcharge{  
...  
}
```

Attributs et Méthodes de classe (statiques):

- VARIABLE DE LA CLASSE

- une variable de classe est accessible à partir de tous les objets instances de cette classe ainsi qu'à partir de la classe elle même.
- En java, on déclare des variables de classe en les faisant précéder du mot clé **static**. (**un seul exemplaire pour tout les objets**)

- Méthode de classe

- une méthode de classe sera une méthode accessible directement à partir de la classe et par les objets. (exemple : la méthode main).
- En java, on déclare les méthodes de classe en les faisant précéder du mot clé **static**.

Les méthodes d'une classe peuvent être déclarés **static** ou non. Dans le premier cas, on dira qu'il s'agit d'une **méthode de classe** et dans le second cas d'une **méthode d'instance**.

Appel d'une méthode de classe

NomDeClasse.NomMéthodeClasse();

Exemple de la classe point avec une méthode fct déclarée statique:

Point.fct();

Tableaux et chaines de caractères en JAVA

Tableaux

Def: C'est une structure de données contenant des éléments de même type, en java les tableaux sont des objets

Déclaration et création d'un tableau

Déclaration:

`type_elements[ ] Nom_tableau ;` ou `type_elements Nom_tableau[ ] ;`

Exemple : `int [ ] T ;` `ou` `int T[];`

Création: (création d'un objet)

`Nom_tableau = new type [dimension] ;`

Exemple : `int [ ] T;`
 `T=new int[5] ;`

Déclaration et Création en une seule instruction:

Comme les autres objets, les tableaux peuvent être déclarés et définis sur la même ligne :

int[] x = new int[5];

Valeurs par défaut dans les composantes après création.

```
int [] tb = new int[3];
```

```
char [] ch = new char[4];
```

```
boolean [] bt = new boolean[3];
```

tb → 0 0 0

ch → '\0' '\0' '\0' '\0'

bt → false false false

Remarques:

- la déclaration ne doit pas indiquer la dimension **int t[5]; (erreur)**

La dimension est donnée avec l'opérateur new qui convoqué à l'exécution contrairement aux autres langages structurés

Autres façons de définir un tableau: à l'initialisation

On peut définir un tableau avec littéral au moment de sa déclaration (sans ajouter new)

Exemple :

`int [] tab = {1,9,2,4};` 

Ou en général:

`int n,p;`

.....

`int [] tab = {1,n,n+p,4};` //Déclaration et initialisation en même temps

Attention ceci est incorrect:

`int [] tab;`

`tab = {1,n,n+p,4};`

Remarques:

- 1) *Même si la dimension d'un tableau est connu au moment de l'exécution le tableau est de taille fixe (tableaux statiques)*
- 2) *Pour les tableaux dynamiques, vous pouvez utiliser la classe vector (java.util)*

Longueur d'un tableau

*On peut connaître la longueur d'un tableau en utilisant l'attribut **length**, **Tableau.length** est la longueur du tableau Tableau.*

```
class TableauB {  
    public static void main( String[ ] argv) {  
        int tableau[ ] = new int[3];  
        System.out.println("taille du tableau : "+ tableau.length) ;  
        for (int i=0 ; i<tableau.length; i++){  
            .....    } } }
```

Dépassement des bornes d'un tableau

Le dépassement des bornes d'un tableau est une erreur non pas de compilation mais d'exécution.

Exemple :

For (int i=0 ; i<=tableau.length ; i++)

Affectation de tableaux:

C'est une Affectation de références

Exercice1:

- Écrire un programme principal qui :
- lit dans un tableau 5 valeurs doubles fournies au clavier
- L'affiche à l'écran

Tableaux en arguments ou en retour à une méthode

Transmission d'un nom d'un tableau à une méthode entraine le passage par référence ce qui permet d'agir directement sur le tableau et non pas sur une copie

Exercice:

Écrire une classe nommée `essai` qui comporte les méthodes statiques:

- 1) `affiche()` qui reçoit un tableau et permet d'afficher ses composantes
- 2) `suite()` qui reçoit un entier `n` et retourne un tableau formés des `n` nombres (1, 2,3,.....`n`)
- 3) `raz()` qui reçoit un tableau d'entiers et remis toutes ses composantes à zéro

Écrire une classe principale de test

Exercice

Réaliser une classe permettant de manipuler des vecteurs ayant un nombre quelconque de composantes de type double. On y prévoira :

- Un constructeur vecteur (int n), n représentant le nombre de composantes qui seront lues au clavier,
- Une méthode (statique) somme fournissant la somme de deux vecteurs ; s'ils n'ont pas la même taille, on renverra une référence "nulle",
- Une méthode "affiche" affichant les composantes d'un vecteur.
- Ecrire un programme d'utilisation

Tableau multidimensionnel

C'est un tableau de tableau.

les tableaux de java peuvent comporter plusieurs dimensions.

Syntaxe de déclaration:

type [] [] [] tableau

Exemple :

int [] [] T ; ou int T[] []

La définition du tableau peut être effectuée à l'aide de :

Tableaux littéraux :

int [] [] tableau = {{1,2},{3,4,5},{5,6},{7,8,9,2}}

Ou à l'aide de :

*l'opérateur new : exemple:***int [] [] x=new int [4][2] ;**

Remarque: *Les lignes n'ont pas forcément le même nombre d'éléments*

Exemple: $\begin{pmatrix} & 2 & 4 \\ 5 & 6 & 1 & 2 \\ & 3 & 0 & 9 \end{pmatrix}$

```
float x[ ][ ] = new float[3][ ];
```

```
x[0]=new float[2]
```

```
x[1]=new float[4]
```

```
x[2]=new float[3]
```

Pour accéder à tous les éléments du tableau, on doit utiliser des boucles imbriquées.

```
for (int i=0; i<x.length; i++) // x.length/ Nombre de lignes de la matrice
{ for (int j=0; j<x[i].length; j++)// x[i].length dimension de la ligne i
{
    x[i][j] = i+j;
}}
```

EXERCICE

Écrire un programme qui :
permet de lire dans une matrice $M(3 \times 5)$ des
nombres entiers fournies au clavier
L'affiche à l'écran

Exercice

Réaliser une classe permettant de manipuler des matrices ayant un nombre quelconque de composantes de type double. On y prévoira :

Un constructeur vecteur (int L, int C), L, et C représentant le nombre de lignes et de colonnes qui seront lues au clavier,

Une méthode d'instance *somme()* fournissant la somme de deux matrices ; s'ils n'ont pas la même taille, on renverra une référence "nulle",

Une méthode "affiche" affichant la matrice.

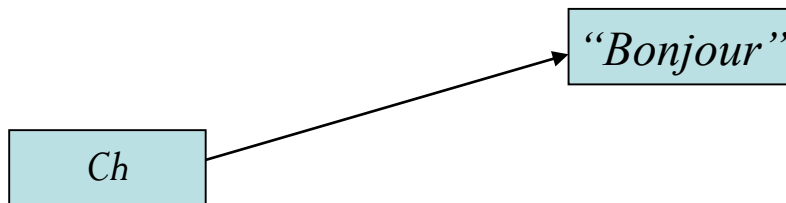
Ecrire un petit programme d'utilisation.

Les chaînes de caractères

- En java, les chaînes de caractères sont des objets.
- Les chaînes instances de la classe **java.lang.String**. ne peuvent pas être modifiées.
- Si on veut traiter de chaînes de caractères modifiables, on peut utiliser la classe **java.lang.StringBuffer** (ou `StringBuilder`: depuis JDK1.5).

Chaine constante

- Comme toute déclaration d'une variable objet, l'instruction :
String ch ; déclare que *ch* est destinée à contenir une référence à un objet de type *String*.
- Ainsi, avec **ch = "bonjour"** ; on aboutit à une situation qu'on peut schématiser ainsi :
- C'est une *chaine constante*, sa référence est créée automatiquement par le compilateur



- La classe *String* dispose de **plusieurs constructeurs**:

Exemple :

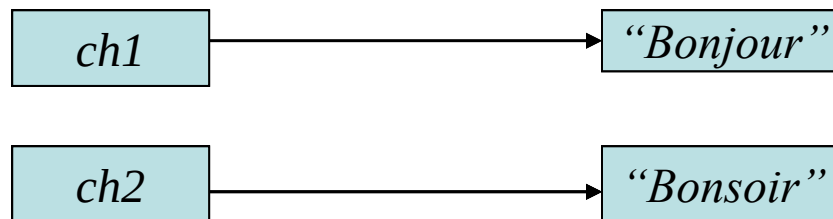
- `String ch1 = new String () ;` // ch1 contient la référence a une chaine vide
- `String ch2 = new String("hello") ;` // ch2 contient la référence a une chaine
// contenant la suite "hello"
- `String ch3 = new String(ch2) ;` // ch3 contient la référence a une chaine
// **copie** de ch2, donc contenant "hello"

Un objet de type String n'est pas modifiable

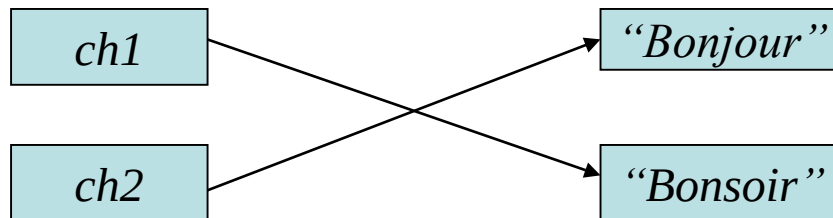
Il n'existera donc aucune méthode permettant d'en modifier la valeur.

Mais on peut modifier les références aux chaînes.

- Considérons ces instructions :
 - String ch1, ch2, ch ;
 - ch1 = "bonjour" ;
 - ch2 = "bonsoir" ;



- Exécutons maintenant ces instructions :
 - `ch = ch1 ;`
 - `ch1 = ch2 ;`
 - `ch2 = ch ;`
- Nous obtenons ceci :



Les deux objets de type chaîne n'ont pas été modifiés, mais les références `ch1` et `ch2` l'ont été.

Entrées-sorties de chaînes

Affichage:

- Nous avons déjà vu qu'on pouvait afficher des constantes chaînes par la méthode *println* :

`System.out.println ("bonjour") ;`
- Elle peut être utilisée de cette manière :
 - `String ch ;`
 - `.....`
 - `System.out.println(ch) ;`

Lecture d'une chaîne au clavier

- utiliser la classe Scanner.

Exemple:

```
import java.util.Scanner
class Entrée-chaîne
{ String ch;
  Scanner obj=new Scanner(System.in)
  ch = obj.nextLine();
}
```

Les principales fonctionnalités de la classe *String*:

- longueur,
- accès aux caractères d'une chaîne par leur position,
- concaténation,
- recherche d'occurrences de caractères ou de sous-chaînes,
- conversions entre types primitifs et type *String*.

*Longueur d'une chaîne : méthode: **length()***

- La méthode **length()** permet d'obtenir la longueur d'une chaîne,

```
String ch = "bonjour" ;  
int n = ch.length() ; // n contient 7
```

Remarque

- Contrairement à ce qui se passait pour les tableaux où **length** désignait un attribut, nous avons bien affaire ici à une **méthode**. Les parenthèses à la suite de son nom sont donc indispensables.

Accès aux caractères d'une chaîne : méthode `charAt()`

- La méthode `charAt` de la classe `String` permet d'accéder à un caractère de rang donné d'une chaîne (le premier caractère porte le rang 0). Ainsi, avec:
 - `String ch = "bonjour" ;`
 - `ch.charAt(0)` correspond au caractère `'b'`,
 - `ch.charAt(2)` correspond au caractère `'n'`.

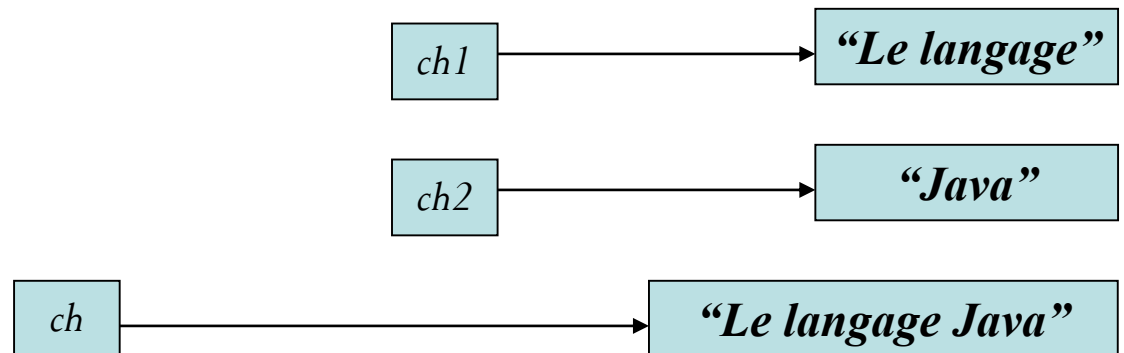
Exercice:

Ecrire un programme principal qui lit une chaîne au clavier et l'affiche verticalement, c'est-à-dire à raison d'un caractère par ligne :

```
import java.util.Scanner
public class Test
{ public static void main (String args[])
    { String mot ;
      Scanner obj=new Scanner(System.in);
      System.out.print ("donnez un mot : ") ;
      mot = obj.nextLine() ;
      System.out.println ("voici votre mot en colonne :") ;
      for (int i=0 ; i<mot.length() ; i++)
      System.out.println (mot.charAt(i)) ;
    }
```

Concaténation de chaînes

- Considérons ces instructions :
 - `String ch1 = "Le langage " ;`
 - `String ch2 = "Java" ;`
 - `String ch = ch1 + ch2 ;`
- Elles correspondent à ce schéma :



Concaténation de chaînes avec des expressions de type primitif.

Java vous autorise également à mélanger chaînes et expressions d'un type primitif.

- **Nous avons déjà utilisé cette possibilité** dans des instructions telles que :

```
int n = 20 ;
```

```
System.out.println ("n = " + n) ; // affiche : n = 20
```

- Ce résultat est obtenu par la **conversion implicite** de la valeur de l'entier *n* en une chaîne.

Comparaisons de chaînes:

Les opérateurs == et !=

On peut comparer deux string à l'aide de l'opérateur == comme dans:

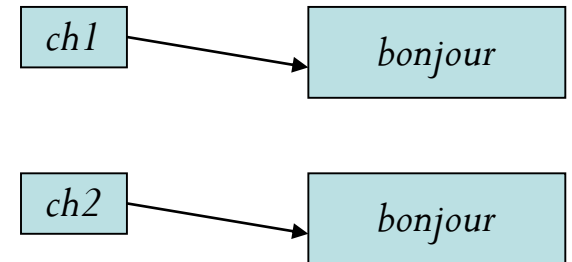
if(str1==str2) ...

or, cette comparaison, bien que correcte, ne compare pas si les deux chaînes sont égales

- Soit par exemple:

```
String ch1 = new String("bonjour ");  
String ch2 = new String("bonjour ");  
if (ch1 == ch2)  
    System.out.println ("egales") ;  
else  
    System.out.println ("differentes") ;
```

// resultat : differentes



La méthode *equals()*

- la classe *String* dispose d'une méthode *equals* qui compare le contenu de deux chaînes

```
String ch1 = new String("bonsoir");
```

```
String ch2 = new String("bonjour");
```

```
.....
```

```
ch1.equals(ch2) // cette expression retourne false
```

```
ch1.equals("bonsoir") // cette expression retourne true
```

- La méthode *equalsIgnoreCase()* :**

Effectue la même comparaison, mais sans distinguer les majuscules des minuscules :

```
String ch1 = "HeLlo" ;
```

```
String ch2 = "hello" ;
```

```
.....
```

```
ch1.equalsIgnoreCase(ch2) // vraie
```

La méthode compareTo()

Permet d'effectuer des comparaisons lexicographiques (la quelle des deux apparaît avant l'autre)

Ch1.compareTo(Ch2);

Retourne:

un entier négatif si Ch1 précède(arrive avant) Ch2

un entier nul si Ch1 égale à Ch2

un entier positif si Ch1 succède(arrive après) Ch2

Exemple: *(Voir Serie de TP)*

Lecture et tri par permutation d'un tableau de string

Modification de chaînes (dans une nouvelle chaîne)

*Les objets de type **String** ne sont pas modifiables.*

Mais, la classe *String* dispose de quelques méthodes qui **créent une nouvelle chaîne** obtenue par transformation de la chaîne courante.

Remplacement de caractères

- La méthode *replace* crée une chaîne en remplaçant toutes les occurrences d'un caractère donné par un autre. Par exemple :

```
String ch = "bonjour" ;
```

```
String ch1 = ch.replace('o', 'a') ; // ch n'est pas modifiée
```

```
// ch1 contient "banjaur"
```


Conversions entre chaînes et types primitifs

- Nous avons déjà vu comment **l'opérateur +** effectuait un "formatage" en convertissant n'importe quel type primitif en une chaîne d'une **manière implicite**.
- Vous pouvez effectuer directement (**explicitement**) de telles conversions, en utilisant la **méthode statique**: *valueOf* de la classe *String*.
- De plus, il est possible, dans certains cas, d'effectuer la conversion en recourant à des méthodes (*toString()*) des **classes enveloppes des types primitifs**.

Conversion d'un type primitif en une chaîne

- Dans la classe *String*, la méthode statique *valueOf* est définie avec un argument des différents types primitifs. Par exemple, pour le type *int* :
int n = 427 ;
String ch = String.valueOf(n) ;
// fournit une chaîne ici "427"

Utilisation de la méthode **toString()** définit dans les **classes enveloppes des types primitifs**.

l'expression :

```
int m=45;  
Integer n=new Integer(m);  
n.toString() ;
```

Les classes enveloppes types primitifs

Classe	Role
String	pour les chaines de caractères
Integer	pour les valeurs entières (integer)
Long	pour les entiers long signés (long)
Float	pour les nombres à virgules flottante (float)
Double	pour les nombres à virgule flottante en double précision (double)
Byte	

Les conversions d'une chaîne en un type primitif

(Méthode statique *parseInt* de la classe enveloppe)

- Par exemple, pour convertir une chaîne en un entier de type *int*, on utilisera la méthode statique *parseInt* de la classe enveloppe *Integer*, comme ceci :
 - `String ch = "3587" ;`
 - `int n = Integer.parseInt(ch) ;`
- D'une manière générale, on dispose des méthodes suivantes :
 - *Byte.parseByte*,
 - *Short.parseShort*,
 - *Integer.parseInt*,
 - *Long.parseLong*,
 - *Float.parseFloat*,
 - *Double.parseDouble*.

Exercice d'application (classe String)

Écrire un programme java permettant de tester les différentes méthodes de la classe String comme suit:

- 2) Lire deux chaînes s1 et s2 et afficher leurs longueurs ainsi que les résultats renvoyés par les expressions
 - s1 == s2
 - s1. equals (s2)
 - s1. compareTo (s2)
 - s1. compareToIgnoreCase (s2)
 - afficher si deux chaînes commencent-elles par le même caractère
 - concaténer s2 à s1. Utilisez l'opérateur + ou la méthode concat.
- 3) Lire une chaîne s1 et:
 - remplacer toutes les instances de caractère o par O (méthode replace).
 - l'afficher entièrement en majuscules (méthode toUpperCase).
- 4) Lire une chaîne s1 comportant plusieurs noms de villes séparés par un et un seul (blanc). Décomposer la chaîne pour accéder directement à chacun des noms de ville grâce à la méthode split. **public String[] split(String regex)**

String[] T= str.split(" ");

Afficher un nom de ville par ligne.

La classe StringBuffer

- Les objets de type *String* ne sont pas directement modifiables, il faut utiliser la classe *StringBuffer*
- Il existe des méthodes :
 - de modification d'un caractère de rang donné : ***setCharAt()***,
 - d'accès à un caractère de rang donné : *charAt*,
 - d'ajout d'une chaîne en fin : la méthode ***append()*** accepte des arguments de tout type primitif et de type *String*,
 - d'insertion d'une chaîne en un emplacement donné : ***insert()***,
 - de remplacement d'une partie par une chaîne donnée : ***replace()***,
 - de conversion de *StringBuffer* en *String* : ***toString()***.

- Voici un programme utilisant ces différentes possibilités :

```
class TstStB
```

```
{ public static void main (String args[])
```

```
{ String ch = "la java" ;
```

```
    StringBuffer chBuf = new StringBuffer (ch) ;
```

```
    System.out.println (chBuf) ;
```

```
    chBuf.setCharAt (3, 'J'); System.out.println (chBuf) ;
```

```
    chBuf.setCharAt (1, 'e') ; System.out.println (chBuf) ;
```

```
    chBuf.append (" 6") ; System.out.println (chBuf) ;
```

```
    chBuf.insert (3, "langage ") ; System.out.println (chBuf) ;
```

```
    } }
```

- la java
- la Java
- le Java
- le Java 6
- le langage Java 6

Exercice d'application (classe StringBuffer)

Écrire un programme java permettant de tester les différentes méthodes de la classe StringBuffer comme suit:

1) Utiliser le constructeur suivant pour créer des StringBuffer :

```
public StringBuffer ( String str)
```

2. Utiliser les méthodes pour modifier votre chaîne :
append, insert, setCharAt, reverse, . . .

L'HERITAGE

Déclaration de l'héritage

*En java, pour déclarer l'héritage, on utilise le mot clé **extends***

Syntaxe :

Class B extends A

- B hérite de A
- B est dérivée de A
- B est une sous classe de A
- A est la super classe de B
- A est la classe de base

L'HERITAGE

Exemple:

Soit la classe *Point* avec un constructeur à 2 arguments.

```
class Point{  
Private float x,y;  
Public Point(float x,float y)  
    {this.x=x; this.y=y;}  
public void affiche( )  
    {system.out.println ("je suis en "+ x + " " +y);}  
...  
}
```

*On voudrait réaliser une autre classe **PointNom** qui hérite de la classe **Point** et qui possède un constructeur à 3 arguments.*

L'HERITAGE

```
class PointNom extends Point
{
    String nom;
    PointNom (float x, float y, String s){
        super(x,y); // appel du constructeur à 2 arg de Point, le mot-clé super.
        nom=s;
    }
}
```

Remarque: l'appel de **super** doit obligatoirement être **la 1ere instruction**.

L'HERITAGE

Redéfinition de méthodes

Une classe dérivée peut en effet fournir une nouvelle définition d'une méthode d'une classe ascendante. Cette fois, il s'agira non seulement de méthode de même nom , mais aussi de même signature et de même type de valeur de retour.

L'HERITAGE

Exemple de la fonction d'affiche:

```
class point
{ private int x,y;
  public point (int x, int y)
  { this.x=x; this.y=y;}
public void affiche()
  {system.out.println (“je suis en ”+ x + “ ” +y);
}
```

L'HERITAGE

```
class PointNom extends Point  
{  
String nom;  
Public PointNom(float x, float y, String s) {  
super(x,y); // appel du constructeur à 2 arg de Point, le mot-clé super.  
nom=s;  
}  
public void affiche( )  
{ super.affiche( ); // et non pas affiche();  
system.out.println ("et mon nom est : "+nom);  
}  
...  
}
```

Exercice:

Réécrire l'exemple complet de la classe point et pointnom (avec un nouveau attribut de type StringBuffer) avec les constructeurs, méthodes affiche et déplace sachant que la fonction déplace de la classe pointnom permet de remplacer le dernier caractère du nom par un caractère passé en argument...

```

class point{
private float x,y;
public point(float x,float y){this.x=x;this.y=y;}
public void affiche(){System.out.print("x="+x+" y="+y);}
public void deplace(float dx,float dy){x+=dx;y+=dy;}}

class pointcar extends point{
private StringBuffer nom;
public pointcar(float x,float y,StringBuffer nom){
super(x,y);
this.nom=nom;}
public void affiche(){super.affiche();
System.out.print("nom="+nom);}
void deplace(float dx,float dy,char c){super.deplace(dx, dy);
int n=nom.length()-1;
nom.setCharAt(n,c);}}

```

```

public class exercice1 {

public static void main(String[] args) {
StringBuffer ch=new StringBuffer("B1");
pointcar a=new pointcar(1,2,ch);
a.affiche();
a.deplace(1, 1, 'J');
a.affiche();}}

```

L'HERITAGE

Les droits d'accès

Si les membres de la classe de base sont :

- **public** ou « **rien** » : les membres de la classe dérivée auront accès à ces membres (champs et méthodes),
 - **private** : les membres de la classe dérivée n'auront pas accès aux membres privés de la classe de base.
-
- En plus des 3 niveaux de protection (public, « rien » et private), il existe un 4e niveau de protection :
 - **protected**. Un membre de la classe de base déclaré **protected** est accessible à ses classes dérivées

Les droits d'accès d'une méthode redéfinie de la classe dérivée doit être plus élevé de celui de la classe de base

```
class A {  
    Public void f(int n) { ... };  
    .... }
```

```
class B extends A {  
    Private void f(int n) { ... };  
    .... }
```

Erreur de compilation

Affectation entre objet de base et objet dérivé

Polymorphisme:

- Le polymorphisme est un concept extrêmement puissant en P.O.O, qui complète l'héritage.

*Soit f une méthode de la classe de base et redéfinie dans la classe dérivée le choix du code à exécuter (pour une méthode polymorphe) ne se fait pas statiquement à la compilation mais **dynamiquement à l'exécution***

Exercice : Exploitez les possibilités de polymorphisme pour créer un tableau “hétérogène” d’objets point et pointnom. Puis afficher le tableau?

```

Public class TABHete
{ public static void main ( String arg[] )
  { Point[ ] tabPts= new Point [4];
    tabPts[0]=new Point(0,2);
    tabPts[1]=new Pointnom(1,5, »A1 »);
    tabPts[2]=new Pointcol(2,8, »A2 »);
    tabPts[3]=new Point(1,2);
    for (int i=0; i<tabpts.length;++ )
      tabPts[i].affiche();
  }
}

```

Résultat:

Je suis un point

Mes coordoonees sont : 0 2

Je suis un point de nom A1

Mes coordoonees sont : 1 5

Je suis un point de nom A2

Mes coordoonees sont : 2 8

Je suis un point

Mes coordoonees sont : 1 2

Exercice:

1) Définissez une classe **Forme** composé:

deux attributs **private** **x** et **y** ?

deux constructeurs, le premier constructeur **Forme ()** sans arguments initialise **x=0** et **y=0**, et le deuxième constructeur **Forme (double x, double y)** contient deux arguments,

Une méthode redéfinie **toString**, la méthode **toString** retourne une chaîne de caractère sous la forme "**Position : (" + x + "," + y + ")**"

2) Définissez une classe **Cercle** dérivée de la classe **Forme** composé:

un attribut **private** **rayon**

deux constructeurs, le premier constructeur **Cercle()** sans arguments initialise **rayon=0** et le deuxième constructeur **Cercle (double rayon)**, 1-3

Une méthode **toString**, permet de retourner une chaîne de caractère qui affiche la position de centre et la valeur du rayon.

3) Écrivez une méthode **c1 plusgrand(Cercle c2)** renvoyant **``vrai"** si le cercle **c1** est plus grand que **c2**.

4) Comment écrire une méthode ayant un comportement similaire, mais prenant les deux cercles en argument ?

Écrire un programme de test.

Classes et méthodes finales (en relation avec l'héritage)

- Une *méthode* déclarée *final* ne peut pas être redéfinie dans une classe dérivée
- Une *classe* déclarée *final* ne peut pas être dérivée

Exercice2:

Soient les classes suivantes : Personne, Enseignant, employé, Etudiant, Etudiant_chercheur, Administrateur.

- 1. Donnez et justifiez une arborescence pour ces classes.*
- 2. Où se situeront les attributs suivants : salaire, emploiDuTemps, anneeEtude, nom, sujet_de_thèse, département.*
- 3. Définir les constructeurs de chaque classe.*
- 4. Redéfinir la méthode toString() de la classe Object dans toutes les classes pour pouvoir afficher leurs état dans la classe de test.*

Classe abstraite

C'est une classe non instantiable, (classe de base pour une dérivation)

Syntaxe:

```
abstract class A{  
....}
```

Exemple:

```
abstract class A{  
  Public void f(){...} // le corps peut être définie  
  Public abstract void g(int n); //uniquement la déclaration de la signature  
}
```

On pourra: déclarer **A a**; mais:

*Non pas l'instantiation: **a =new A()**; //erreur compilation*

Exemple

Soit la classe 'Humain', à partir de laquelle dérivent la classe 'Homme' et la classe 'Femme'

'Homme' et 'Femme' sont instanciables (les objets créés ont une existence en soi).

*Cependant, **la classe 'Humain' sera déclarée abstraite** car un objet 'Humain' n'existe pas en tant que tel, puisqu'il manque l'information sur le sexe*

La classe 'Humain' servira à implémenter des méthodes qui seront utilisées à la fois pour 'Homme' et pour 'Femme'

Dérivation d'une classe abstraite

```
class B extends A{  
  Public void g(int n){...} // redéfinition de la méthode abstraite  
  ....  
}
```

On pourra alors instantier B: $B\ a = \text{new } B();$ //OK
A x=B; ou : $A\ a = \text{new } B();$ //OK

Remarques:

1 : *si une classe A contient une méthode abstraite, alors A est **considérée abstraite** sans qu'elle ne soit pas précédée par le mot clé abstract.*

2: *une **méthode abstraite se déclare tjrs public**, et doit obligatoirement être implémentée différemment dans toutes les classes filles*

3: *dans l'entête d'une méthode abstraite le nom de l'identificateur doit apparaître:*

class A

{public abstract void g(int); //erreur : nom d'argument obligatoire

}

Exercice(Création de classes permettant de représenter des figures géométrique en 2D)

Classe cercle: caractérisée par les coordonnées du centre, du rayon et de sa couleur

Classe rectangle: caractérisée par les coordonnées du son coin gauche, sa largeur, de sa longueur et de sa couleur.

Les figures disposent de plusieurs méthodes:

Exemple la méthode abstraite: change-echelle(),

1)Quelles sont les attributs communs aux cercles et aux rectangles?

*2)Définissez une classe abstraite Figure2D qui regroupe les données et traitements communs aux cercles et aux rectangles? Et qui doit redéfinir la méthode **toString()**.*

3) Définissez les classes cercle et rectangle et testez ces classes dans un programme principal?

```
abstract class figure2D{
protected int x,y;
protected StringBuffer couleur;
protected figure2D(int x,int y, StringBuffer couleur){this.x=x;this.y=y;this.couleur=couleur;}
public abstract void chageechelle(int y);
public String toString(){return ("x="+x+"y="+y+"couleur="+couleur);}}
```

```
class rectangle extends figure2D{
int larg,longue;
public rectangle(int x,int y, StringBuffer couleur,int larg,int longue){
super(x,y,couleur);this.larg=larg;this.longue=longue;}
public void chageechelle(int c){larg=larg*c;longue=longue*c;}
public String toString(){
return (super.toString()+"largeur="+larg+"longuer="+longue);}}
```

```
class cercle extends figure2D{
int rayon;
public cercle(int x,int y, StringBuffer couleur,int rayon){
super(x,y,couleur);this.rayon=rayon;}
public void chageechelle(int c){rayon=rayon*c;}
public String toString(){return (super.toString()+"rayon="+rayon);}}
public class jjjj {
public static void main(String[] args) {
StringBuffer ch=new StringBuffer("rouge");
rectangle r0=new rectangle(1,2,ch,5,6);
System.out.println("rectangle"+r0.toString());
// TODO Auto-generated method stub
}}
```

Exercice:

Le but de l'exercice est de créer une hiérarchie de classes pour représenter les étudiants d'une université. Il y a 3 types d'étudiants : ceux en Licence, ceux en Master, et ceux en Doctorat.

- Chaque étudiant a un nom, une adresse et un numéro (code).
 - Les étudiants ont un profil. Pour les étudiants en Licence on parle de parcours. Les étudiants en Master une spécialité et les étudiants en Doctorat un directeur de recherche.
1. Définir les classes nécessaires à cette hiérarchie de classes, en leurs ajoutant les membres nécessaires et les méthodes usuelles (constructeurs, la redéfinition de toString(), etc...) dans chacune des classes.
 2. Écrire un programme de test pour de créer un tableau de type Etudiants (un ensemble d'étudiants) et qui permet d'afficher toutes les informations les concernant

Considérant l'exercice précédent,
Écrire maintenant une classe `ListeEtudiants` ayant un attribut
(tableau d'étudiants) et comporte un constructeur:

public ListeEtudiants(Etudiant[] T) {...}

Une méthode *int compteEtudiants(String type)* qui reçoit un
type d'étudiants et retourne le nombre des étudiants
correspondants

*Utiliser: getClass().getSimpleName() définie dans la classe
Object pour savoir le nom de la classe d'un objet (elle
retourne: Nompacage.nomclasse*

Et aussi compareTo() pour comparer deux chaînes

Ecrire un programme principal de test

Refaire maintenant la classe ListeEtudiants ayant toujours un attribut (tableau d'étudiants) et un indice initialisé à zéro lors de la construction et comporte un constructeur

public ListeEtudiants(int taille) {...}

Une méthode ***public void ajoutEtudiant(Etudiant E){...}***

NB/ Tenez compte de ne pas dépasser la taille de votre tableau

Une méthode AfficheEtudiants()

Ecrire un programme principal de test

Les interfaces

*une interface est une classe abstraite (pure) qui comporte uniquement **les entêtes de méthodes** ainsi que **des constantes**.*

- Toutes les méthodes sont: **public abstract** et **non static***
- Tous les champs sont des constantes : **public static final***

Différences entre Interface et classe abstraite:

*Une interface pourra **hériter** de plusieurs interfaces (héritage multiple), alors qu'une classe abstraite ne pouvait dériver que d'une seule classe abstraite.*

Syntaxe de définition d'une interface:

public interface *NomInterface* {...}

Exemple:

public interface *I* {
void *f*(int *n*); // équivalent à : **public abstract** *Void f*(int *n*);
void *g*();
}

Implémentation des interfaces: (mots clé implements)

Syntaxe:

classe *A implements I*{
//*A* doit redéfinir toutes les méthodes *f* et *g* de l'interface *I*
}

Remarque: une même classe peut implémenter plusieurs interfaces

classe *A implements I1,I2,...*{

A doit redéfinir **toutes** les méthodes de *I1*, et *I2*, et ...

}

*Soit I une interface, la variable i (**I i**) est destinée à contenir une référence à un objet de toute classe implémentant I .*

*classe A **implements** I*

...

}

I $i = \text{new } A(\dots);$

Exercice(Création de classes et interfaces permettant de représenter des figures géométrique en 2D)

Interface Figure2D avec une méthode translation(double hor, double vert) et affiche()
et interface Surface2D avec une méthode calculsurface()

Class point

Classe cercle: caractérisée par le centre (type point) et du rayon

Classe rectangle: caractérisée par son coin gauche(type point), sa largeur, de sa longueur

- 1)Définissez les deux interfaces ainsi que les classes point, cercle et rectangle
- 2) Testez ces classes dans un programme principal?

Héritage d'interface

```
public interface A {  
    public void methodA();  
}
```

```
public interface C extends A{  
    public void methodC();}
```

La classe qui doit implémenter l'interface C doit redéfinir les méthodes de A et de C

Héritage multiple d'interface

Possible (contrairement aux classes)

```
public interface A {  
    public void methodA();  
}
```

```
public interface B {  
    public void methodB();  
}
```

```
public interface AB extends A, B {  
    public void Autremethod();  
}
```

Exercice:

Une entreprise souhaite gérer les salaires et les primes de ses employés selon la hiérarchie de classes suivantes:

(Toutes les classes comportent les méthodes usuelles (constructeurs, la redéfinition de toString(), etc...)

- 1) Employé: classe non instanciable (super-classe de la hiérarchie) qui comporte: le nom, le prénom, l'age et la date d'entrée en service. cette classe comporte en plus une méthode abstraite **calculerSalaire()** (qui dépend du type de l'employé)
- 2) Commercial: classe non instanciable qui hérite de l'employé et comporte un attribut chiffreAffaire, une méthode getchiffreAffaire(), etc
- 3) Vendeur: classe qui hérite de commercial et comporte une constante pourcent = 0.2 % du chiffre d'affaire
- 4) Représentant: classe qui hérite de commercial et comporte une constante pourcent = 0.1 % chiffre d'affaire,
- 5) technicien qui hérite de l'employé et comporte un salaire_UNITE fixé à 50 DH et un nombre d'unités,
- 6) Manutentionnaire qui hérite de l'employé et comporte un SALAIRE_HORAIRE fixe 100 DH et un nombre d'heurs
- 7) TechnicienRisque qui hérite du technicien et qui a un risque de réparation
- 8) ManutRisque qui hérite du Manutentionnaire et qui a un risque de fabrication
- 9) une interface risque (prime=500)

Calcul du salaire

Vendeur: salaire mensuel est le 20 % du chiffre d'affaire qu'ils réalisent,

Représentant: salaire mensuel est également le 10 % du chiffre d'affaire qu'ils réalisent,

technicien: salaire vaut le nombre d'unités produites mensuellement multipliées par salaire_Unité.

Manutentionnaire: salaire vaut leur nombre d'heures de travail mensuel multipliées par salaire_horaire.

4) écrire une classe principale de test qui permet de créer un tableau des différents employés et de les afficher

À partir du *Java.8: depuis* mars 2014

La notion d'interface est profondément modifiée en Java 8

À partir de Java 8, on peut ajouter deux éléments supplémentaires dans une interfaces :

des *méthodes statiques* et des *méthodes par défaut*.

Une méthode par défaut: permet d'écrire une méthode dans une interface, en fixant sa signature et en donnant une implémentation

Syntaxe:

```
default void f(){...}
```

```
interface TestInterface
{
    // abstract method
    public void square(int a);

    // default method
    default void show()
    {
        System.out.println("Default Method Executed");
    }
}
```

```
class TestClass implements TestInterface
{
    // implementation of square abstract method
    public void square(int a)
    {
        System.out.println(a*a);
    }

    public static void main(String args[])
    {
        TestClass d = new TestClass();
        d.square(4);

        // default method executed
        d.show();
    }
}
```

```
public interface A {  
default void f() { // implémentation } }
```

```
public interface B {  
default void f() { // implémentation } }
```

```
public class C implements A, B {  
// !!! erreur de compilation !!! // corps de la  
classe }
```

```
interface TestInterface
{
    // méthode abstraite
    public void square (int a);

    // méthode statique
    static void show()
    {
        System.out.println("Static Method Executed");
    }
}
```

```
class TestClass implements TestInterface
{
    // Implementation de la méthode abstraite
    public void square (int a)
    {
        System.out.println(a*a);
    }

    public static void main(String args[])
    {
        TestClass d = new TestClass();
        d.square(4);

        // appel de la méthode statique
        TestInterface.show();
    }
}
```

Nouveautés de java8:

- *Java 8 et les applications graphiques natives*
- *Java 8 et les objets connectés*
- *Java 8 et le cloud (serveurs accessible via internet)*
- *Java 8 et les applications mobiles*

LES EXCEPTIONS

Mécanisme de gestion des erreurs dans un programme Java

Définition d'une exception:

une exception est une interruption de l'exécution d'un programme suite à une erreur.

*Par exemple, une division par zéro provoque une exception de type **ArithmeticException**.*

*Alors conduit à **l'arrêt du programme**.*

Un premier exemple d'une exception

Avant de voir comment traiter les exceptions, regardons à quoi elles correspondent en prenant un exemple très simple de division par zéro :

```
class DivParZero{  
    public static void main (String[] argv)  
    {  
        int zero = 0;  
        zero = 200/zero;  
        System.out.print("et je continu!!!");  
    }  
}
```

A l'exécution du programme, on obtient l'affichage suivant:

Exception in thread "main" java.lang.ArithmeticException: / by zero at DivParZero.main(DivParZero.java:6)

LES EXCEPTIONS

Il faut alors gérer ces exceptions pour éviter l'arrêt du programme

Mécanisme:

- 1) Lancer l'exception: en utilisant le mot clé throws
- 2) Attraper une exception: try{...}
- 3) Gérer l'exception catch(){...}

Attraper et gérer une exception

*Pour pouvoir attraper une exception il faut entourer le code susceptible de la lancer dans un bloc précédé de l'instruction **try**.*

LES EXCEPTIONS

Les blocs try et catch

- Le mot clé **try** permet de spécifier un bloc de code sur le quel on s'attend qu'une exception soit levée (c à d possibilité d'avoir une erreur).
- Le bloc de l'instruction **try** doit être suivi d'une instruction **catch**
- Le mot clé **catch** sert à spécifier le code à exécuter pour une exception donnée. Il faut faire suivre le mot **catch** d'une parenthèse ouvrante, d'un type exception (une classe), du nom qu'on lui donnée, d'une parenthèse fermante et du code associé placé entre accolade.

Donc les blocs try et catch sont utilisés comme suit :

```
Try { //zone contenant des instructions pouvant générer des erreurs( lever des exception)}  
catch ( NomException e)  
{  
    // Traitement à faire dans le cas de l'exception e  
}
```

*un bloc **try** peut être suivi (**de manière contigu**) de plusieurs blocs catch chacun spécifiant un type d'exception différent :*

```
try {  
    opertaion_dangereuse1 ;  
    opertaion_dangereuse2 ;  
    opertaion_dangereuse3 ;  
}  
catch (Exception1 e1)  
{... }  
catch (Exception2 e2)  
{...}  
catch (Exception3 e3)  
{...}
```

```
public class Test {  
    public static void main(String[] args) {  
        Int x,y;  
        Scanner sc= new Scanner(System.in);  
        System.out.println("Veuillez saisir deux entiers");  
        int x = sc.nextInt(); int y = sc.nextInt();  
        try { System.out.println(x / y); }  
        catch (Exception e) {  
            System.out.println("Exception : Division par zéro "); }  
        System.out.println(« et je continu"); } }
```

Et si vous ne connaissez pas le type d'exception

```
public class Test {  
    public static void main(String[] args) {  
        Int x,y;  
        Scanner sc= new Scanner(System.in);  
        System.out.println("Veuillez saisir deux entiers");  
        int x = sc.nextInt(); int y = sc.nextInt();  
        try { System.out.println(x / y); }  
        catch (Exception e) {  
            System.out.println("Exception : Division par zéro "); }  
  
        System.out.println(« et je continu"); } }
```

Utiliser des methodes de la classe: Exception

```
public class Test {  
    public static void main(String[] args) {  
        Int x,y;  
        Scanner sc= new Scanner(System.in);  
        System.out.println("Veuillez saisir deux entiers");  
        int x = sc.nextInt(); int y = sc.nextInt();  
        try { System.out.println(x / y); }  
        catch (Exception e) {  
            System.out.println("Exception :"+ e.getMessage()); }  
  
        System.out.println(« et je continu"); } }
```

Exception : / by zero
et je continu

LES EXCEPTIONS

Quelques classes d'exceptions prédéfinis

Exception	Type d'exception
IOException	Entrée Sortie Exemples: lecture sur un fichier inexistant
ArithmeticException	Arithmétique Exemple: Division par zéro
ArrayIndexOutOfBoundsException	Dépassement des bornes d'un tableau
NumberFormatException	Format du nombre n'est pas respectée Exemple : un int représentée en float
OutOfMemoryException	Dépassement de mémoire

Définir une exception

- *Si on veut pouvoir signaler un événement exceptionnel d'un type non prévu par l'API*
il faut étendre la classe `java.lang.Exception`.

Exemple: cette classe peut être utilisée comme exception dans une autre classe:
Exemple :

```
Class MaPropreException extends Exception{ }
```

Lancer l'exception: en utilisant le mot clé throws

Généralement, l'exception lancée n'est pas traitée par la méthode elle-même, il est obligatoirement que l'entête de la méthode signale le lancement de l'exception:

Exemple:

```
Class erreur extends Exception{ }
```

```
Class point{  
Public point (int x,int y) throws erreur{  
If(x<0 || y<0) throw new erreur();  
This.x=x; this.y=y;}  
}
```

Syntaxe:

```
Type_retour f(arguments)throws Nom_Exception{.....}
```

Le lancement d'une exception, (à l'aide du mot réservé **throw**), on crée une instance de la classe définie.

```
public class test {  
  
public static void main(String[] args) {  
try{  
point a=new point(-1,2);  
}  
catch(erreur e){System.err.println("Attention erreur");}  
  
}  
System.out.println("et je continue ");  
}
```

Un objet instance d'une exception (erreur dans notre cas) peut appeler une méthode de la classe Exception

D'où le test suivant:

```
public class test {  
  
    public static void main(String[] args) {  
        try{  
            point a=new point(-1,2);  
        }  
        catch(erreur e){System.err.println(e.getMessage());}  
  
    }  
}
```

*Résultat d'affichage: **null***

```

class erreur extends Exception{
    erreur(String mess){super(mess);}
}
class point{
    private float x,y;
    public point(float x,float y)throws erreur{
        if(x<0 ||y<0) throw new erreur("Attention erreur");
        this.x=x;this.y=y;}
}
public class chap_exeception {
    public static void main(String[] args) {
        try{
            point a=new point(-1,2);
        }
        catch(erreur e){System.err.println(e.getMessage());}
        //catch(Exception e){System.err.println(e.getMessage());}
    }
    System.out.println("et je continue ");}

```

Attention erreur
et je continue

Exercice

Soit la classe Point avec un constructeur à 2 arguments et une méthode d'affichage.

Réaliser une autre classe cercle qui hérite de la classe Point et qui possède un nouveau attribut rayon. La classe comporte un constructeur à 3 arguments (il faut vérifier que la valeur de son rayon est positive. Dans le cas contraire, une exception sera levée) et la redéfinition de la méthode affiche().

Écrire un programme de test qui permet de créer et d'afficher un tableau de 4 composantes (deux points et deux cercles). Le programme doit attraper et traiter toutes les exceptions.

Exercice2

Ecrire une classe compteBancaire: Un compte bancaire est identifié par un numéro de compte, Nom et prénom de la personne ayant ce compte, le solde du compte **qui doit être positif ou nul à la création(si non lancement d'une exception)**.

Le numéro du compte **ne doit pas dépasser une certain taille si non lancement d'une exception**

Une méthode Débiter pour retirer un montant positif au solde du compte (**lancement d'une exception(votre propre exception affichant un message passé à son constructeur si le solde est insuffisant)**)

Classe Abstraite en C++

Classe contenant **au moins une méthode abstraite**.

Méthode abstraite: *méthode virtuelle pure (sans code)*

Déclaration d'une classe abstraite: *Il n'y a pas de mot-clé spécifique.*

```
class B {  
    virtual void affiche() [const] = 0;  
    ... };
```

```
class Figure {  
    public: virtual double surface() const = 0;  
           virtual double perimetre() const = 0;  
};
```

```
class Cercle : public Figure{
```

```
...
```

```
public: virtual double surface() const { return M_PI * r * r; }
```

```
virtual double perimetre() const {return 2 * M_PI * r; } };
```

```
class Rectangle : public Figure {
```

```
...
```

```
public: virtual double surface() const { return L * l; } };
```

Exercice(En C++):

Une entreprise souhaite gérer les salaires et les primes de ses employés selon la hiérarchie de classes suivantes:

(Toutes les classes comportent les méthodes usuelles (constructeurs, la méthode d'affichage et sa redéfinition, etc...))

- 1) Employé: classe non instanciable (super-classe de la hiérarchie) qui comporte: le nom, le prénom, l'age et la date d'entrée en service. cette classe comporte en plus une méthode abstraite **calculerSalaire()** (qui dépend du type de l'employé)
- 2) Technicien qui hérite de l'employé et comporte un salaire_UNITE fixé à 50 DH et un nombre d'unités,
- 6) Manutentionnaire qui hérite de l'employé et comporte un SALAIRE_HORAIRE fixe 100 DH et un nombre d'heurs
- 7) TechnicienRisque qui hérite du technicien et qui a un risque de réparation
- 8) ManutRisque qui hérite du Manutentionnaire et qui a un risque de fabrication
- 9) une classe abstraite pure risque (prime=500)

Calcul du salaire

Vendeur: salaire mensuel est le 20 % du chiffre d'affaire qu'ils réalisent,

Représentant: salaire mensuel est également le 10 % du chiffre d'affaire qu'ils réalisent,

technicien: salaire vaut le nombre d'unités produites mensuellement multipliées par salaire_Unité.

Manutentionnaire: salaire vaut leur nombre d'heures de travail mensuel multipliées par salaire_horaire.

- 4) écrire une classe principale de test qui permet de créer un tableau des différents employés et de les afficher avec leurs salaires

Exercice (à faire en C++ et en JAVA) : héritage et tableaux d'objets

On veut modéliser la gestion d'une bibliothèque : on définira un certain nombre de classes : **Documents, Livre et dictionnaires**.

Tous les documents possèdent un numéro d'enregistrement, un titre et nombre d'exemplaires. A chaque livre est associé, en plus, un auteur et un nombre de pages, les dictionnaires ont, en plus, comme attributs une langue et un nombre de tomes.

- 1) Définissez les classes suivantes : Documents, Livre et dictionnaires.
Pour chacune Définissez un constructeur permettant d'initialiser toutes ses variables d'instances. Une méthode d'affichage.

La classe Documents possède un attribut de classe pour attribuer automatiquement un numéro à chaque document créé.

2) Définissez une classe `listeDocuments` contenant :

- Un tableau d'objets documents comme attribut qui peut contenir soit des livres ou bien des dictionnaires.
- Un constructeur permettant de créer le tableau vide et avec une taille maximale passé en paramètre,
- Une méthode `void Ajoutdocument(Documents d)` qui permet d'ajouter un livre ou un dictionnaire au tableau.
- Une méthode `void Listedocuments()` qui permet d'afficher le code, le nombre d'exemplaires et le titre des documents contenus dans le tableau.
- Une méthode `int comptedocument(String type)` qui compte et retourne le nombre de livres, par exemple, qui se trouve dans le tableau. (vous devez utiliser la méthode `getClass().getSimpleName()` de la classe `Object` pour savoir le nom de classe de l'objet qui fait appel à cette méthode)

Ecrire une classe de test pour pouvoir :

- Créer 2 livres et 2 dictionnaires*
- Les ajouter dans le tableau d'objets défini dans la classe `listeDocuments`.*
- Afficher tous les documents.*
- Afficher le nombre de livre et le nombre de dictionnaire disponibles dans la bibliothèque*